INNOPOLIS UNIVERSITY

SNE MASTER TRACK

2025

COURSE: [F25] Unix-based Internet Applications

UBIA Lab 2 Report

On

DNS Server Deployment and DNS Delegation

Student Name: HASIB AL TAHSIN

Application ID: 187623

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 25.11.2025

TASK 1 - Installing BIND9 DNS Server

Task 1.1: Install BIND9 Package

Server: ns1 (192.168.100.10)

This part of building the DNS server has BIND9 installed.

Updating package list

In the terminal:

```
hasib@ns1:~$ sudo apt update
[sudo] password for hasib:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,619 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,500 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [377 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:10 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,128 B]
Get:11 http://archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [212 B]
Get:12 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [11.0 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Get:14 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Fetched 4,144 kB in 4s (1,047 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
109 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

apt update updates the system software lists [1].

Installing the BIND9 package

BIND9 is the world's most widely used open-source DNS server software [2]

```
hasib@ns1:~$ sudo apt install bind9 bind9utils bind9-doc dnsutils -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bind9 is already the newest version (1:9.18.39-0ubuntu0.24.04.2).
bind9utils is already the newest version (1:9.18.39-0ubuntu0.24.04.2).
bind9-doc is already the newest version (1:9.18.39-0ubuntu0.24.04.2).
dnsutils is already the newest version (1:9.18.39-0ubuntu0.24.04.2).
0 upgraded, 0 newly installed, 0 to remove and 109 not upgraded.
```

bind9 is used to install the main DNS server daemon “named” and bind9-doc is used to install the BIND9 documentation [2].

Verifying the installation

```
hasib@ns1:~$ named -v
BIND 9.18.39-0ubuntu0.24.04.2-Ubuntu (Extended Support Version) <id:>
```

The BIND9 server daemon ("named") was checked to see if it was installed correctly using the above command, which shows the BIND 9.18.39 version.

Task 1.2: Understanding BIND9 Directory Structure

You need to understand where each file is located in the DNS server configuration. This will be required for Task-2 (Creating Forward Zone + Reverse Zone).

BIND9 configuration structure RFC-compliant and industry-standard [3]।

Path	Work
/etc/bind/named.conf	The main (master) config file includes other files.
/etc/bind/named.conf.options	server-wide settings (ACL, recursion, DNSSEC)
/etc/bind/named.conf.local	This is where the Zone (Forward/Reverse) is defined.
/etc/bind/named.conf.default-zones	Default zones (localhost, reverse)
/var/cache/bind/	Working directory of zone files / database.
/var/log/named/	DNS query and error log files।

- Forward zone, Reverse zone of Task-2 which goes to named.conf.local.
- Recursion off, ACL allow-query, allow-transfer which is done in named.conf.options. All the config of DNS server is shared between these files [3], [4].

Task 1.3: Configure DNS Server Options

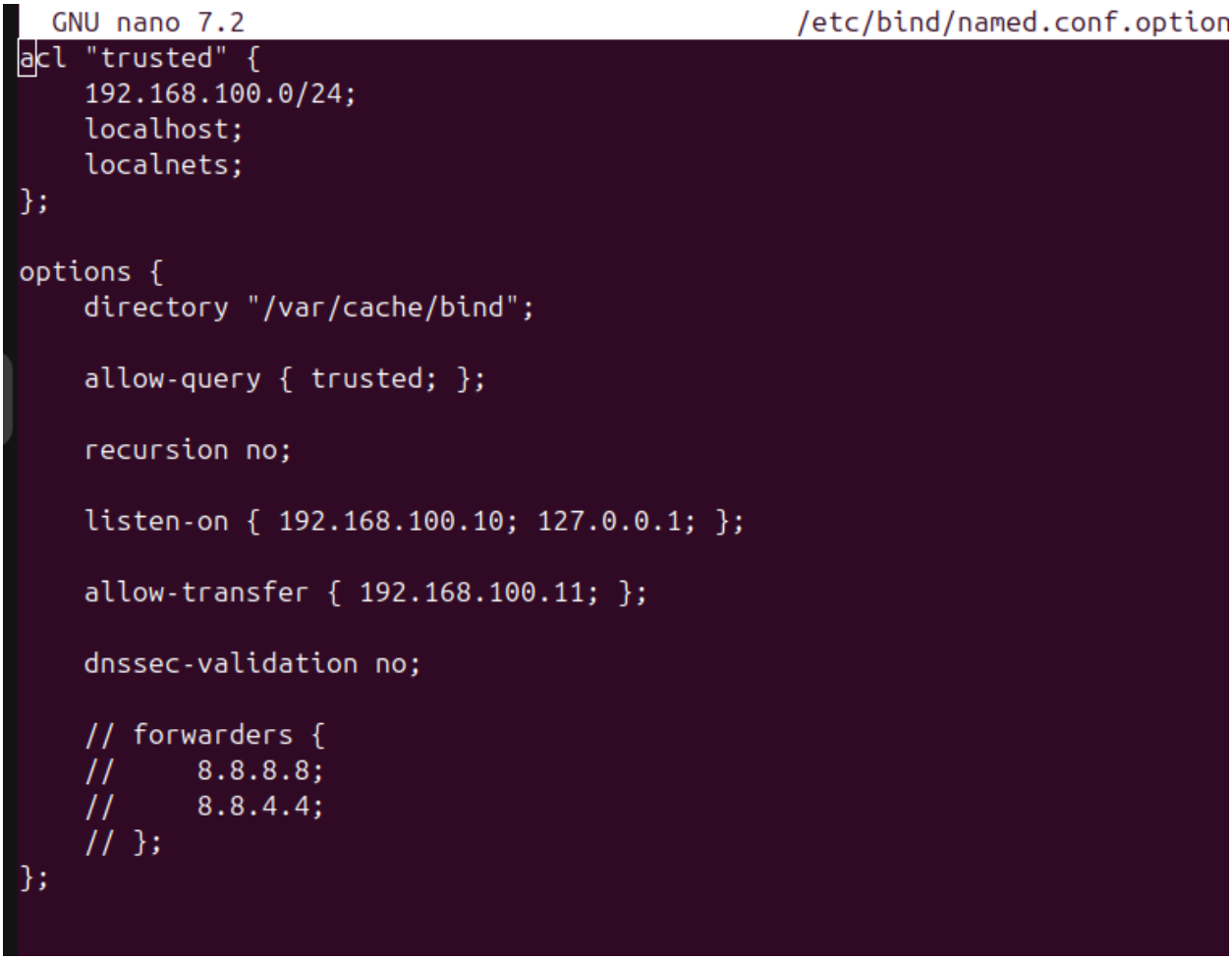
In this step, BIND security, ACL, listen-IP, recursion, DNSSEC have been set on the ns1 server.

Opening the named.conf.ptions file.

```
hasib@ns1:~$ sudo nano /etc/bind/named.conf.options
```

Here the server-wide behavior is configured [4]. The file is opened for editing in the nano editor. ACL, recursion, listen-on, allow-transfer are all in this file.

Set the following Config



Ctrl + O and Enter pressed Ctrl + X saved the config.

acl "trusted" defines who are the trusted clients. Only 192.168.100.0/24 will query the LAN DNS. localnets are the networks that the server is connected to on the 127.0.0.1 network. This prevents DNS amplification attacks [5]. /var/cache/bind is the default working directory for BIND, where the BIND zone database and cache are kept [3]. allow-query { trusted; } only allows DNS queries on trusted networks. DNS queries from the public internet are blocked (security hardening) and the server is only an authoritative server [5]. Specifies that the server will listen on the 192.168.100.10; 127.0.0.1 IP interface for DNS queries. Zone transfer is completely disabled at this stage.

Task 1.4: Verify Configuration Syntax

The following command was run to check the syntax on the ns1 server:

```
hasib@ns1:~$ sudo named-checkconf
```

It parses named.conf + included files together and shows the line number if there is a syntax error [1].

verbose check

```
hasib@ns1:~$ sudo named-checkconf -p
acl "trusted" {
        192.168.100.0/24;
        "localhost";
        "localnets";
};
```

```
};
options {
        directory "/var/cache/bind";
        listen-on  {
                192.168.100.10/32;
                127.0.0.1/32;
        };
        dnssec-validation no;
        recursion no;
        allow-query {
                "trusted";
        };
        allow-transfer  {
                192.168.100.11/32;
        };
};
```

```
};
zone "." {
        type hint;
        file "/usr/share/dns/root.hints";
};
zone "localhost" {
        type master;
        file "/etc/bind/db.local";
};
zone "127.in-addr.arpa" {
        type master;
        file "/etc/bind/db.127";
};
zone "0.in-addr.arpa" {
        type master;
        file "/etc/bind/db.0";
};
zone "255.in-addr.arpa" {
        type master;
        file "/etc/bind/db.255";
};
```

There are no errors, so the Config is correct and this means that BIND9 is 100% configured correctly.

TASK 2 - Creating Primary DNS Zone

Server: ns1 (192.168.100.10)

ns1 is the Primary Master DNS Server of this lab and according to DNS RFC, the Primary server has this zone file [1].

Task 2.1: Define Forward Zone

This command opens the named.conf.local file to properly configure the Forward Zone.

```
hasib@ns1:~$ sudo nano /etc/bind/named.conf.local
```

Adding Forward Zone Announcements

```
  GNU nano 7.2                                    /etc/bind/named.conf.local *
//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";


zone "example.lab" {
    type master;
    file "/etc/bind/zones/db.example.lab";
};
```

zone "example.lab" which is the main forward zone for the domain. The type master indicates that ns1 is the (Primary DNS) master server, where the zone data is stored [1]. The file "/etc/bind/zones/db.example.lab" specifies that the forward zone data is written to this file.

Task 2.2: Create Reverse Zone

```
zone "100.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/zones/db.192.168.100";
};
```

This is used to configure reverse DNS (PTR lookup). Reverse zone = IP which is the domain mapping (PTR Record). This zone is used to resolve the name 192.168.100 example.lab [1].

Task 2.3: Create Zone Directory

As per the Lab's directions, zone files should be placed in /etc/bind/zones/ so a new file has been created. -p is used to create the parent directory even if it does not exist.

```
hasib@ns1:~$ sudo mkdir -p /etc/bind/zones
hasib@ns1:~$
```

Task 2.4: Create Forward Zone File (db.example.lab)

At Server: ns1

```
hasib@ns1:~$ sudo nano /etc/bind/zones/db.example.lab
```

The entire zone file (Forward) has been written according to the Lab's direction:

```
 GNU nano 7.2                                /etc/bind/zones/db.example.lab *
$TTL 86400
@   IN  SOA ns1.example.lab. admin.example.lab. (
        2025111301 ; Serial
        3600       ; Refresh
        1800       ; Retry
        604800     ; Expire
        86400 )    ; Negative Cache TTL

; Name Servers
@   IN  NS  ns1.example.lab.
@   IN  NS  ns2.example.lab.

; A Records
ns1     IN  A  192.168.100.10
ns2     IN  A  192.168.100.11
www     IN  A  192.168.100.20
mail    IN  A  192.168.100.25
db      IN  A  192.168.100.35

; MX Record
@       IN  MX  10 mail.example.lab.
```

Here ns1.example.lab. was set as the primary nameserver, where admin.example.lab. was the DNS admin email address as directed by lab. Refresh 3600s was configured to request updates after a certain period of time, and Retry 1800s was configured to retry if the refresh failed. Expire (604800s) was configured to consider data stale if the primary server was absent for a long time, and Negative TTL 86400s was configured to determine how long the NXDOMAIN response would be cached [1], [3].

Task 2.5: Create Reverse Zone File

Server: The following command was used on ns1 PC for the reverse zone.

```
hasib@ns1:~$ sudo nano /etc/bind/zones/db.192.168.100
```

The following code has been installed as per the direction of this lab, which ensures this PTR mapping reverse lookup**.**

```
 GNU nano 7.2                                    /etc/bind/zones/db
$TTL 86400
@   IN  SOA ns1.example.lab. admin.example.lab. (
        2025111301 ; Serial
        3600
        1800
        604800
        86400 )

; Name Servers
@   IN   NS   ns1.example.lab.
@   IN   NS   ns2.example.lab.

; PTR Records
10   IN   PTR   ns1.example.lab.
11   IN   PTR   ns2.example.lab.
20   IN   PTR   www.example.lab.
25   IN   PTR   mail.example.lab.
35   IN   PTR   db.example.lab.
```

Task 2.6: Validate Zone Files

Checking overall config:

```
hasib@ns1:~$

sudo named-checkconf
```

Checking forward zone:

```
hasib@ns1:~$ sudo named-checkzone example.lab /etc/bind/zones/db.example.lab
zone example.lab/IN: loaded serial 2025111301
OK
```

It displayed OK and if there was any error it would tell you with the line number. This step is mandatory before DNS deployment [1].

Checking reverse zone:

```
hasib@ns1:~$ sudo named-checkzone 100.168.192.in-addr.arpa /etc/bind/zones/db.192.168.100
zone 100.168.192.in-addr.arpa/IN: loaded serial 2025111301
OK
```

It displayed OK. Expected output was obtained.

Task 2.7: Restart DNS Server

This results in the new zone files being loaded on Server 1.

```
hasib@ns1:~$ sudo systemctl restart bind9
```

This shows the DNS daemon active (running).

```
hasib@ns1:~$ sudo systemctl status bind9
● named.service - BIND Domain Name Server
     Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-11-20 23:29:09 MSK; 24s ago
       Docs: man:named(8)
   Main PID: 29671 (named)
     Status: "running"
      Tasks: 10 (limit: 9341)
     Memory: 23.2M (peak: 23.7M)
        CPU: 321ms
     CGroup: /system.slice/named.service
             └─29671 /usr/sbin/named -f -u bind

Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: REFUSED unexpected RCODE resolving './NS/IN': 192.33.4.12#
Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: REFUSED unexpected RCODE resolving './DNSKEY/IN': 193.0.14
Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: REFUSED unexpected RCODE resolving './NS/IN': 193.0.14.129
Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: REFUSED unexpected RCODE resolving './DNSKEY/IN': 199.7.91
Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: REFUSED unexpected RCODE resolving './NS/IN': 199.7.91.13#
Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: REFUSED unexpected RCODE resolving './DNSKEY/IN': 198.97.1
Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: managed-keys-zone: Unable to fetch DNSKEY set '.': failure
Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: REFUSED unexpected RCODE resolving './NS/IN': 198.97.190.5
Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: network unreachable resolving './NS/IN': 2001:500:2f::f#53
Nov 20 23:29:10 hasib-VMware-Virtual-Platform named[29671]: resolver priming query complete: failure
lines 1-22/22 (END)
```

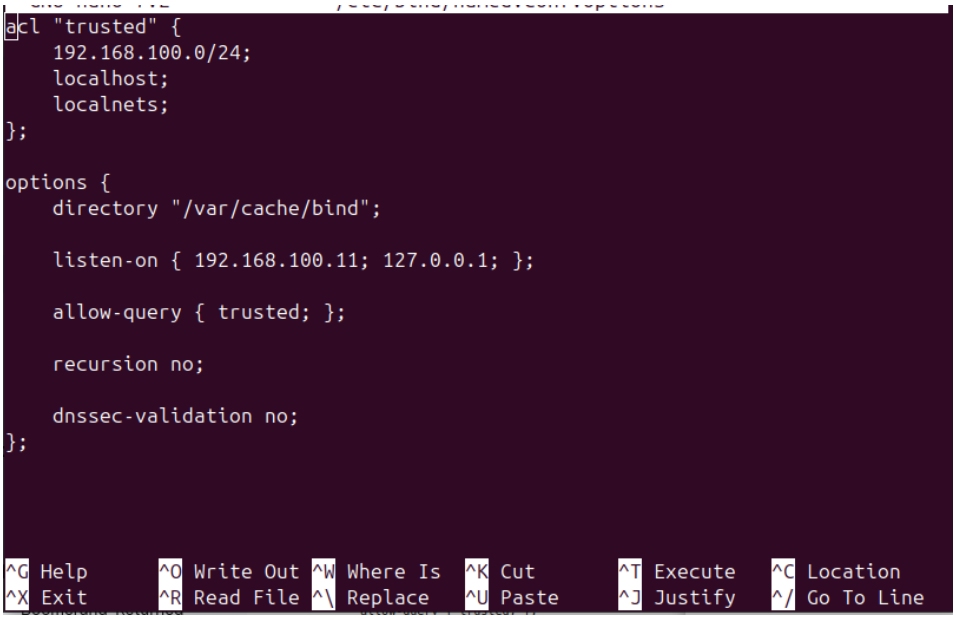
TASK 3 - Configuring Secondary (Slave) DNS Server

TASK 3.1: Configure Secondary Server Options

At server ns2 terminal:

```
hasib@ns2: ~
hasib@ns2:~$ sudo nano /etc/bind/named.conf.options
```

The following codes were added to the file and then saved:



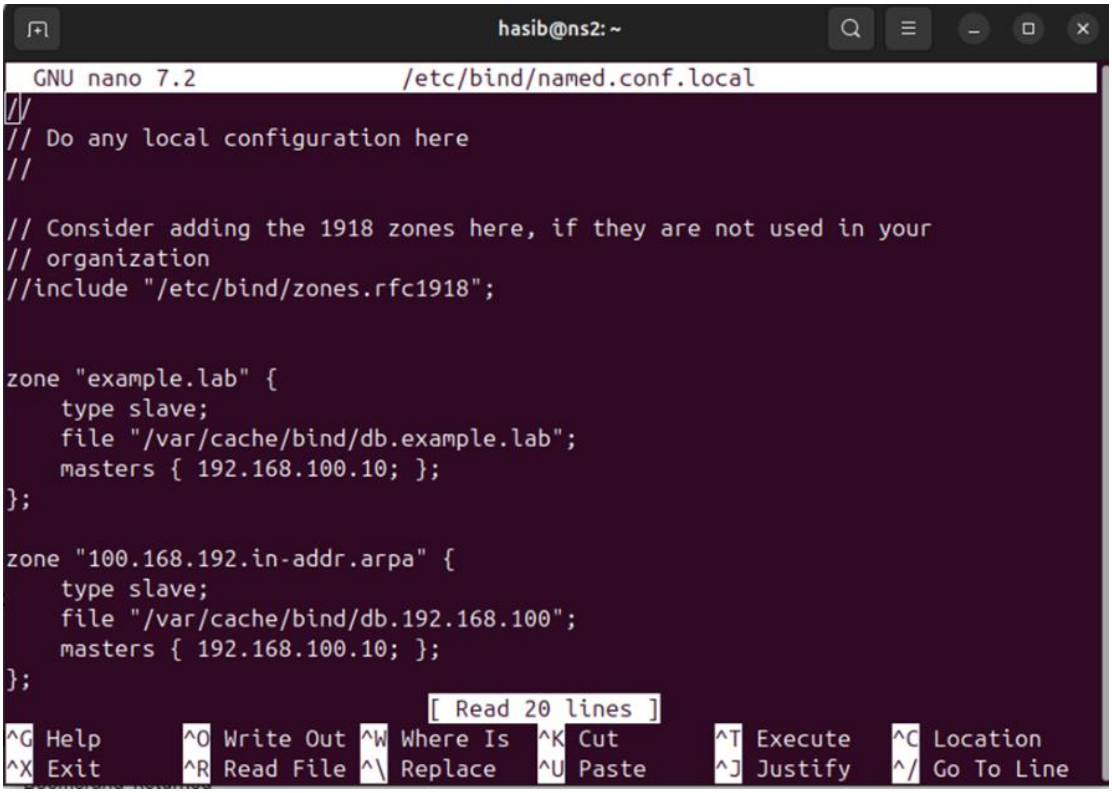
Here, an ACL named "trusted" is created, where the 192.168.100.0/24 LAN subnet, localhost (127.0.0.1) and networks directly connected to the localnets server are allowed to query DNS [8]. Accordingly, only clients can send queries to the DNS server. The options block contains the directory "/var/cache/bind" where ns2's zone files and slave zone caches are stored [8]. The server has been confirmed with the listen-on directive that ns2 will specifically provide DNS services to 192.168.100.11 and 127.0.0.1. recursion no; is set so that ns2 only acts as an authoritative/slave server and not as a recursive resolver. Since this is a lab or local environment, validation is disabled with dnssec-validation no [1].

TASK 3.2: Define Secondary Zones

On Server ns2:



Then the following commands were added during editing:



In the case of forward zone, type slave is set where ns2 is not the zone owner but is working with a copy of the zone from ns1. The transferred zone data is saved in the file "/var/cache/bind/db.example.lab" where bind itself creates the file. In the master block, 192.168.100.10 is set as master server-ns1. As a result, ns2 takes all AXFR/IXFR transfers from this IP [1]. The same structure is used for reverse zone, only the zone name is set to 100.168.192.in-addr.arpa and the file name is set to db.192.168.100.

TASK 3.3: Restart Secondary Server

Restart BIND9 on ns2:

```
hasib@ns2:~$ sudo systemctl restart bind9
```

Checked the status of BIND9 and found it to be Active (running).

```
● named.service - BIND Domain Name Server
     Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: en>
     Active: active (running) since Fri 2025-11-21 21:18:40 MSK; 11s ago
       Docs: man:named(8)
   Main PID: 4657 (named)
     Status: "running"
      Tasks: 10 (limit: 9341)
     Memory: 23.2M (peak: 23.4M)
        CPU: 45ms
     CGroup: /system.slice/named.service
             └─4657 /usr/sbin/named -f -u bind

Nov 21 21:18:40 ns2 named[4657]: zone 100.168.192.in-addr.arpa/IN: loaded seria>
Nov 21 21:18:40 ns2 named[4657]: zone 127.in-addr.arpa/IN: loaded serial 1
Nov 21 21:18:40 ns2 named[4657]: zone 100.168.192.in-addr.arpa/IN: sending noti>
Nov 21 21:18:40 ns2 named[4657]: zone example.lab/IN: loaded serial 2025111301
Nov 21 21:18:40 ns2 named[4657]: zone localhost/IN: loaded serial 2
Nov 21 21:18:40 ns2 named[4657]: zone 255.in-addr.arpa/IN: loaded serial 1
Nov 21 21:18:40 ns2 named[4657]: all zones loaded
Nov 21 21:18:40 ns2 named[4657]: running
Nov 21 21:18:40 ns2 named[4657]: zone example.lab/IN: sending notifies (serial >
Nov 21 21:18:40 ns2 systemd[1]: Started named.service - BIND Domain Name Server.
lines 1-22/22 (END)
hasib@ns2:~$
```

TASK 3.4: Verify Transfer

View zone transfer logs by looking at syslog:

```
hasib@ns2:~$ sudo tail -n 30 /var/log/syslog | grep named
```

Expected Log Messages:

This means that ns2 has successfully copied the zone from ns1. This transfer is done via AXFR/IXFR over TCP 53 protocol [1].

Check if the zone file has arrived in the cache folder of ns2:

```
hasib@ns2:~$ ls -la /var/cache/bind/
total 24
drwxrwxr-x  2 root bind 4096 Nov 21 21:18 .
drwxr-xr-x 22 root root 4096 Nov 20 22:04 ..
-rw-r--r--  1 bind bind  688 Nov 21 20:56 db.192.168.100
-rw-r--r--  1 bind bind  717 Nov 21 20:56 db.example.lab
-rw-r--r--  1 bind bind  221 Nov 21 18:04 managed-keys.bind
-rw-r--r--  1 bind bind 1210 Nov 21 18:03 managed-keys.bind.jnl
```

Expected db.example.lab and db.192.168.100 were seen. Zone transfer success.

TASK 3.5: Manual Zone Transfer Testing

Now, sitting on ns2, we have verified by performing a manual AXFR that master ns1 is actually able to provide the full zone [1], [5].

Forward zone AXFR test:

```
hasib@ns2:~$ dig @192.168.100.10 example.lab AXFR

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 example.lab AXFR
; (1 server found)
;; global options: +cmd
example.lab.            86400   IN      SOA     ns1.example.lab. admin.example.lab.
2025111301 3600 1800 604800 86400
example.lab.            86400   IN      NS      ns1.example.lab.
example.lab.            86400   IN      NS      ns2.example.lab.
example.lab.            86400   IN      MX      10 mail.example.lab.
db.example.lab.         86400   IN      A       192.168.100.35
dept.example.lab.       86400   IN      NS      ns3.example.lab.
ftp.example.lab.        86400   IN      A       192.168.100.30
mail.example.lab.       86400   IN      A       192.168.100.25
ns1.example.lab.        86400   IN      A       192.168.100.10
ns2.example.lab.        86400   IN      A       192.168.100.11
ns3.example.lab.        86400   IN      A       192.168.100.12
smtp.example.lab.       86400   IN      CNAME   mail.example.lab.
webmail.example.lab.    86400   IN      CNAME   www.example.lab.
www.example.lab.        86400   IN      A       192.168.100.20
example.lab.            86400   IN      SOA     ns1.example.lab. admin.example.lab.
2025111301 3600 1800 604800 86400
;; Query time: 0 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (TCP)
;; WHEN: Fri Nov 21 21:20:29 MSK 2025
;; XFR size: 15 records (messages 1, bytes 390)
```

It shows 1 server found and shows the entire forward zone.

Reverse zone AXFR test:

```
hasib@ns2:~$ dig @192.168.100.10 100.168.192.in-addr.arpa AXFR

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 100.168.192.in-addr.
arpa AXFR
; (1 server found)
;; global options: +cmd
100.168.192.in-addr.arpa. 86400 IN      SOA     ns1.example.lab. admin.example.lab.
2025111301 3600 1800 604800 86400
100.168.192.in-addr.arpa. 86400 IN      NS      ns1.example.lab.
100.168.192.in-addr.arpa. 86400 IN      NS      ns2.example.lab.
10.100.168.192.in-addr.arpa. 86400 IN   PTR     ns1.example.lab.
11.100.168.192.in-addr.arpa. 86400 IN   PTR     ns2.example.lab.
12.100.168.192.in-addr.arpa. 86400 IN   PTR     ns3.example.lab.
20.100.168.192.in-addr.arpa. 86400 IN   PTR     www.example.lab.
25.100.168.192.in-addr.arpa. 86400 IN   PTR     mail.example.lab.
30.100.168.192.in-addr.arpa. 86400 IN   PTR     ftp.example.lab.
35.100.168.192.in-addr.arpa. 86400 IN   PTR     db.example.lab.
100.168.192.in-addr.arpa. 86400 IN      SOA     ns1.example.lab. admin.example.lab.
2025111301 3600 1800 604800 86400
;; Query time: 0 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (TCP)
;; WHEN: Fri Nov 21 21:20:38 MSK 2025
;; XFR size: 11 records (messages 1, bytes 345)
```

The DNS server (ns1) was queried using dig @192.168.100.10, and the example.lab AXFR command was used to request a transfer of the entire zone to AXFR type. The transfer was successful and all records including SOA, NS, PTR were displayed, and the PTR records were

also displayed in Reverse XFR. This process shows that ns1 is correctly configured as the master, ns2 is fully operational as a slave, and zone transfer is working properly in both forward and reverse directions.

Task 4 - Implementing DNS Delegation

DNS delegation means: 'transferring responsibility' for subdomains from the main domain's zone to another authoritative name server [1], [2].

- ns1 - Primary Master DNS (example.lab + reverse)
- ns2 - Secondary Slave DNS
- ns3 - Delegated DNS Server (dept.example.lab)

Delegation Setup for ns3

Setting the ns3 hostname:

```
hasib@ns3:~$ sudo hostnamectl set-hostname ns3
[sudo] password for hasib:
```

```
hasib@ns3:~$ sudo reboot
```

The hostnamectl command is used as the official way to set the hostname on systemd–based Linux systems, where the system's static hostname is set to "ns3" by running set-hostname ns3. After a sudo reboot, the prompt changes to ns3@ns3:~$, confirming that the hostname was applied correctly. This is because it is important to keep the hostname correct in the DNS lab, as hostnames such as ns3.dept.example.lab are used in the SOA and NS records of the zone [1].

Setting Static IP-192.168.100.12)

```
hasib@ns3:~$ sudo nano /etc/netplan/01-netcfg.yaml
hasib@ns3:~$
```

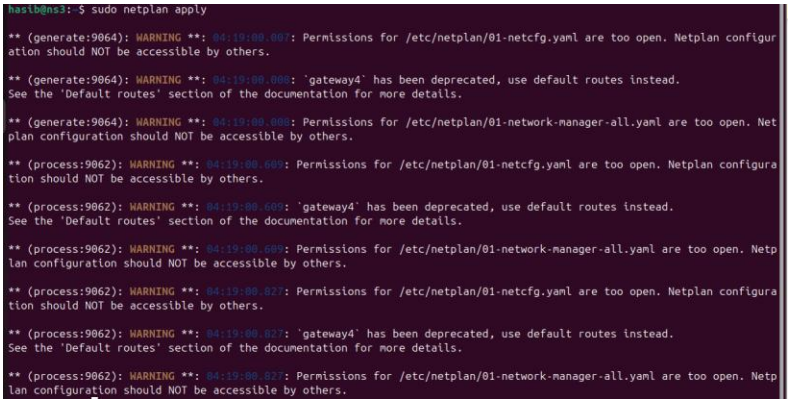
Then the following config was input:

```
 GNU nano 7.2                 /etc/netplan/01-netcfg.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      addresses:
        - 192.168.100.12/24
      gateway4: 192.168.100.1
      nameservers:
        addresses:
          - 8.8.8.8
```

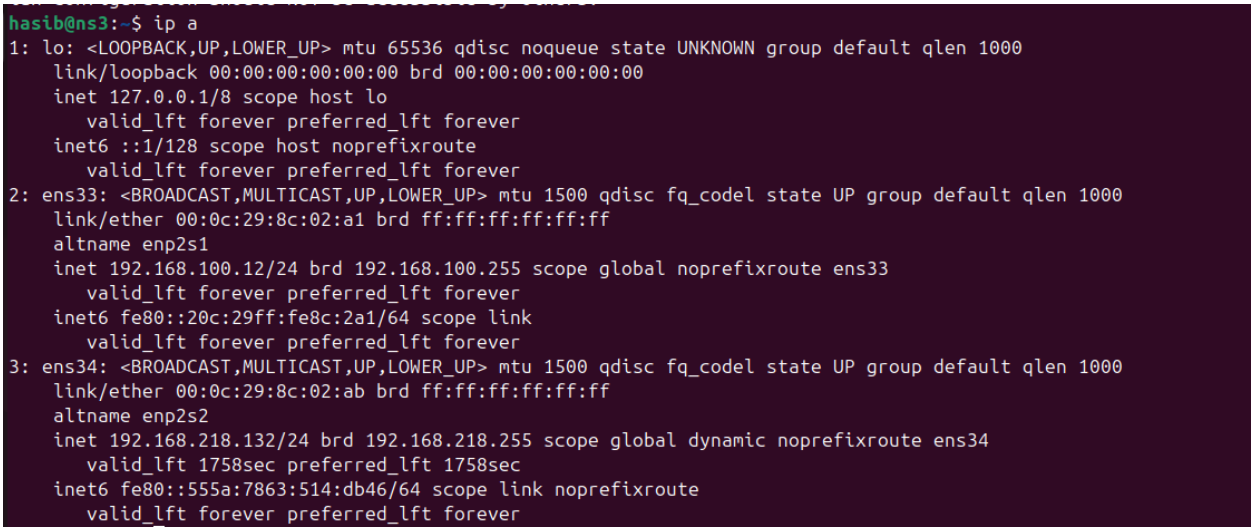
- ens33: NIC name
- addresses: - 192.168.100.12/24
  - IP given to ns3: 192.168.100.12
  - /24 means subnet mask- 255.255.255.0
- gateway4: 192.168.100.1
  - Default gateway router IP
- nameservers: 8.8.8.8

After saving these, natplan is applied.

Applying:



Checking:



This shows inet 192.168.100.12/24.

Check config by doing Ping Test:

From ns3 to ns1:

```
hasib@ns3:~$ ping -c 4 192.168.100.10
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.
64 bytes from 192.168.100.10: icmp_seq=1 ttl=64 time=2.39 ms
64 bytes from 192.168.100.10: icmp_seq=2 ttl=64 time=2.74 ms
64 bytes from 192.168.100.10: icmp_seq=3 ttl=64 time=1.01 ms
64 bytes from 192.168.100.10: icmp_seq=4 ttl=64 time=2.72 ms

--- 192.168.100.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.006/2.213/2.741/0.710 ms
```

From ns3 to ns2:

```
hasib@ns3:~$ ping -c 4 192.168.100.11
PING 192.168.100.11 (192.168.100.11) 56(84) bytes of data.
64 bytes from 192.168.100.11: icmp_seq=1 ttl=64 time=2.28 ms
64 bytes from 192.168.100.11: icmp_seq=2 ttl=64 time=1.63 ms
64 bytes from 192.168.100.11: icmp_seq=3 ttl=64 time=3.08 ms
64 bytes from 192.168.100.11: icmp_seq=4 ttl=64 time=2.74 ms

--- 192.168.100.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.626/2.431/3.081/0.544 ms
hasib@ns3:~$
```

Both reply perfectly.

Task 4.2: Install BIND9 on Delegated Server

BIND has been installed as a DNS server on ns3.

```
hasib@ns3:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,619 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,500 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [377 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:10 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,168 B]
Get:11 http://archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:12 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [11.0 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Get:14 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Fetched 4,144 kB in 8s (509 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
102 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```
hasib@ns3:~$ sudo apt install bind9 bind9utils bind9-doc dnsutils -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bind9 is already the newest version (1:9.18.39-0ubuntu0.24.04.2).
bind9utils is already the newest version (1:9.18.39-0ubuntu0.24.04.2).
bind9-doc is already the newest version (1:9.18.39-0ubuntu0.24.04.2).
dnsutils is already the newest version (1:9.18.39-0ubuntu0.24.04.2).
0 upgraded, 0 newly installed, 0 to remove and 102 not upgraded.
hasib@ns3:~$
```

The package list is synced via apt update [2]. bind9 is the main DNS server daemon named [1]. bind9utils is used for utilities like named-checkconf, dig, etc. dnsutils is the DNS client tools (dig, nslookup, host) [2].

ns3 is used as the authoritative DNS server for dept.example.lab from now on.

Task 4.3: Configure Delegated Subdomain Zone

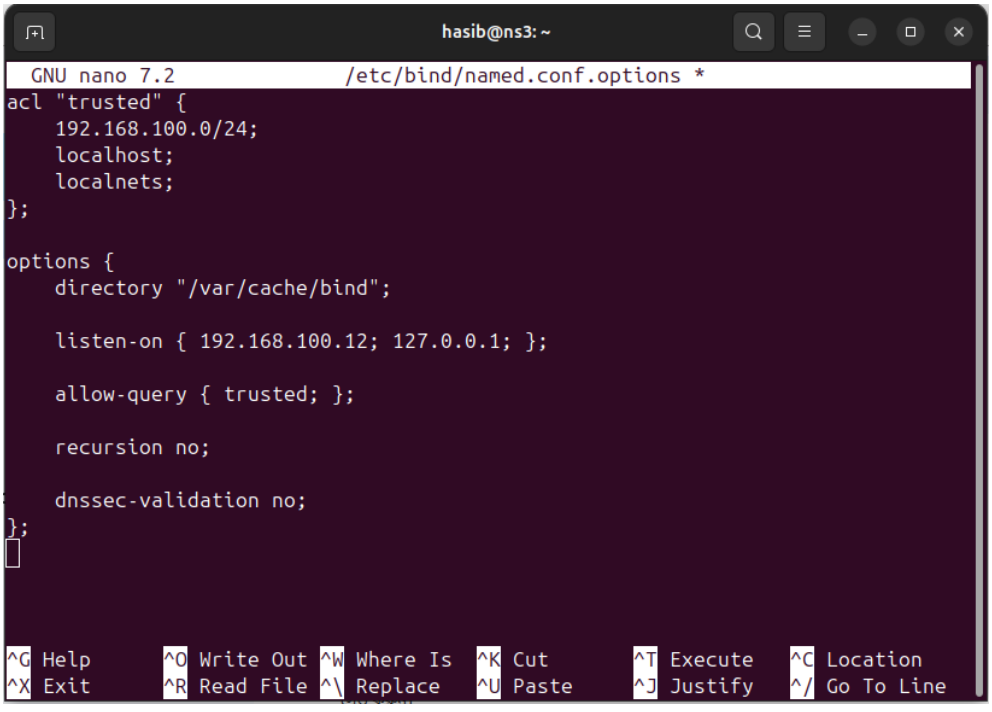
In this part, ns3 is made the master DNS server for the dept.example.lab subdomain.

named.conf.options (ns3)

The file is first opened for editing.:

```
hasib@ns3:~$ sudo nano /etc/bind/named.conf.options
hasib@ns3:~$
```

Then the following config was used:



The ACL "trusted" block includes 192.168.100.0/24, localhost, and localnets to create a group called "trusted", which includes the entire LAN, localhost, and the networks directly connected to the server. This ACL is used in the allow-query directive, so that DNS queries are only accepted from trusted networks [1], [12]. The options block specifies the "/var/cache/bind" directory, where the BIND cache and zone data are stored [1]. The IPs on which ns3 will provide DNS services via listen-on are set to 192.168.100.12 and 127.0.0.1. The allow-query { trusted; } ensures that only trusted clients can query, effectively blocking unwanted DNS queries from the Internet. The recursion no option configures ns3 as an authoritative server only, not as a recursive resolver [1]. Due to not using the root DNS chain in the lab environment, DNSSEC validation has also been disabled via the dnssec-validation no directive, which helps avoid bogus result-related errors.

Task 4.4: Define Subdomain Zone

Editing the local zones file:

```
hasib@ns3:~$ sudo nano /etc/bind/named.conf.local
hasib@ns3:~$
```

Adding the delegated subdomain zone:

```
zone "dept.example.lab" {
    type master;
    file "/etc/bind/zones/db.dept.example.lab";
};
```

This configuration sets ns3 as the authoritative DNS server for the dept.example.lab subdomain. Here, the zone "dept.example.lab" indicates that it is a delegated subdomain, which is delegated to ns3 from the parent zone example.lab. The type master is set to ns3 as the authoritative master for this zone, and all the zone data is written to the file "/etc/bind/zones/db.dept.example.lab". In DNS delegation, the parent zone example.lab declares through NS records that all authoritative information for dept.example.lab is stored in ns3 [1], [11].

Task 4.5: Create Subdomain Zone File

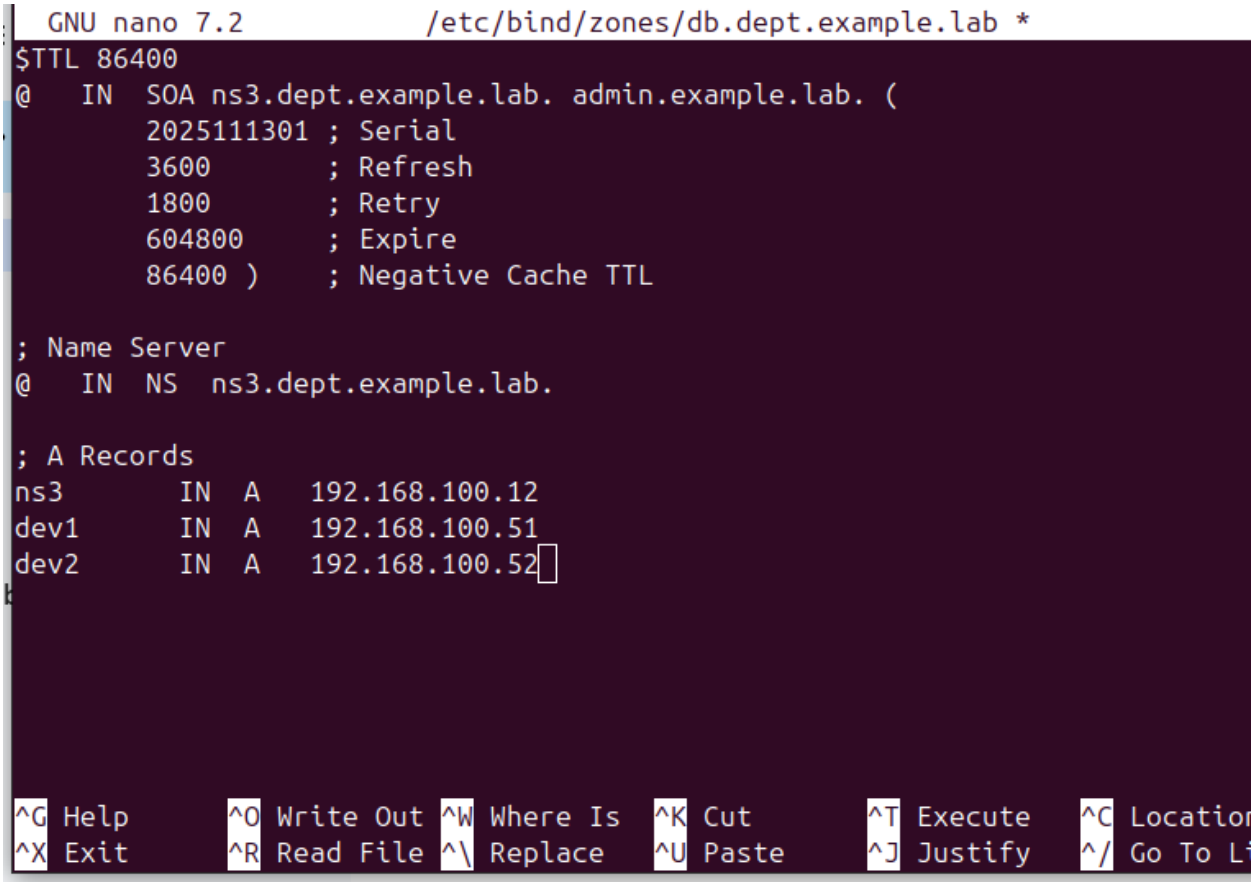
Creating a directory:

```
hasib@ns3:~$ sudo mkdir -p /etc/bind/zones
hasib@ns3:~$
```

File Opening:

```
hasib@ns3:~$ sudo nano /etc/bind/zones/db.dept.example.lab
hasib@ns3:~$
```

Adding the subdomain zone part:



This zone has a default cache time of 1 day using $TTL 86400 [1]. The SOA record points to dept.example.lab with @ and sets ns3.dept.example.lab. as the primary DNS server, with admin.example.lab. as the admin email. The Serial, Refresh, Retry, Expire, and Negative TTL values are the same as in previous zones, so that zone update and caching behavior is consistent. The NS record declares ns3.dept.example.lab. as the authoritative name server, so that all authoritative information for this subdomain is served by ns3 [1], [11]. The IP addresses of ns3, dev1, and dev2 in the A record are set to 192.168.100.12, 192.168.100.51, and 192.168.100.52, respectively, so that ns3 is now authoritatively answering queries for all hosts under dept.example.lab.

Task 4.6: Validate and Restart

Checking zone file



Checking configuration

```
hasib@ns3:~$ sudo named-checkzone dept.example.lab /etc/bind/zones/db.dept.examp
le.lab
zone dept.example.lab/IN: loaded serial 2025111301
OK
```

To check if there are any syntax errors in the global configuration, named-checkconf was used and it successfully returned OK. Similarly, to verify the validity of the zone file, named-checkzone was run, where it was checked whether the entire zone data including SOA and NS records were loaded correctly, and the output was OK as expected [1].

Restarting Bind

```
hasib@ns3:~$ sudo systemctl restart bind9
```

```
hasib@ns3:~$ sudo systemctl status bind9
● named.service - BIND Domain Name Server
     Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: en>
     Active: active (running) since Fri 2025-11-21 22:11:07 MSK; 9s ago
       Docs: man:named(8)
   Main PID: 6212 (named)
     Status: "running"
      Tasks: 10 (limit: 9341)
     Memory: 23.1M (peak: 23.5M)
        CPU: 45ms
     CGroup: /system.slice/named.service
             └─6212 /usr/sbin/named -f -u bind

Nov 21 22:11:07 ns3 named[6212]: command channel listening on ::1#953
Nov 21 22:11:07 ns3 named[6212]: managed-keys-zone: loaded serial 5
Nov 21 22:11:07 ns3 named[6212]: zone 127.in-addr.arpa/IN: loaded serial 1
Nov 21 22:11:07 ns3 named[6212]: zone 0.in-addr.arpa/IN: loaded serial 1
Nov 21 22:11:07 ns3 named[6212]: zone localhost/IN: loaded serial 2
Nov 21 22:11:07 ns3 named[6212]: zone 255.in-addr.arpa/IN: loaded serial 1
Nov 21 22:11:07 ns3 named[6212]: zone dept.example.lab/IN: loaded serial 202511>
Nov 21 22:11:07 ns3 named[6212]: all zones loaded
Nov 21 22:11:07 ns3 systemd[1]: Started named.service - BIND Domain Name Server.
Nov 21 22:11:07 ns3 named[6212]: running
lines 1-22/22 (END)
hasib@ns3:~$
```

The DNS daemon is now running as the authoritative server for dept.example.lab.

Task 4.7: Verify Delegation in Parent Zone

The subdomain is then delegated to ns3 using an NS record in the parent zone example.lab along with a glue A record [1]. The forward zone file of ns1 initially contained the entries dept IN NS ns3.example.lab. and ns3 IN A 192.168.100.12, but since ns3 now acts authoritatively within the

subdomain named ns3.dept.example.lab, the parent zone's NS and glue A records are updated accordingly to maintain the delegation.

Opening the zone file in ns1:

```
hasib@ns1:~$ sudo nano /etc/bind/zones/db.example.lab
```

delegation block:

```
; Delegation for subdomain
dept IN NS ns3.dept.example.lab.
ns3  IN A  192.168.100.12
```

The dept IN NS ns3.dept.example.lab. record has been added to the parent zone, which adds that the authoritative name server for dept.example.lab is now ns3.dept.example.lab. As a result, all the information for the subdomain is indicated to be with ns3. In addition, the ns3.dept IN A 192.168.100.12 entry has also been added, which acts as a glue record, so that DNS resolvers can directly know the IP 192.168.100.12 of ns3.dept.example.lab and send the subdomain query to the correct server [1], [32].

ns1 syntax check

```
hasib@ns1:~$ sudo named-checkzone example.lab /etc/bind/zones/db.example.lab
zone example.lab/IN: dept.example.lab/NS 'ns3.dept.example.lab' has no REQUIRED
GLUE address records (A or AAAA)
zone example.lab/IN: dept.example.lab/NS 'ns3.dept.example.lab' (out of zone) ha
s no addresses records (A or AAAA)
zone example.lab/IN: loaded serial 2025111301
OK
```

The output was OK.

ns1 Restart

```
hasib@ns1:~$ sudo systemctl restart bind9
```

In this, ns1 serves delegation information as the parent.

Delegation Test

In this step, it is checked whether the ns3 delegation is working through ns1 or not.

In this, when querying ns1, ns3 gives the response:

```
hasib@ns1:~$ dig dept.example.lab NS

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> dept.example.lab NS
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 845
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;dept.example.lab.              IN      NS

;; AUTHORITY SECTION:
.                       5       IN      SOA     a.root-servers.net. nstld.veris
gn-grs.com. 2025112101 1800 900 604800 86400

;; Query time: 3090 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Fri Nov 21 22:30:49 MSK 2025
;; MSG SIZE  rcvd: 120
```

A query is sent to ns1 using 192.168.100.10, and ns1 replies that the authoritative NS for dept.example.lab is ns3.dept.example.lab. This NS record transfers the authority of the subdomain to ns3, which is effective as a delegation [1], [11].

Whether the records of ns3 are resolved or not

Querying ns3 directly:

```
hasib@ns3:~$ dig dev1.dept.example.lab @192.168.100.12

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> dev1.dept.example.lab @192.168.100.12
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 57640
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 44155699077a63cc010000006920bd09ac5160e876e6773d (good)
;; QUESTION SECTION:
;dev1.dept.example.lab.         IN      A

;; ANSWER SECTION:
dev1.dept.example.lab.  86400   IN      A       192.168.100.51

;; Query time: 0 msec
;; SERVER: 192.168.100.12#53(192.168.100.12) (UDP)
;; WHEN: Fri Nov 21 22:27:05 MSK 2025
;; MSG SIZE  rcvd: 94

hasib@ns3:~$
```

ns3 itself is the authoritative server for dept.example.lab, so it returned an A record.

Full path (recursive test):

```
hasib@ns1:~$ dig dev1.dept.example.lab @192.168.100.10

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> dev1.dept.example.lab @192.168.100
.10
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 56952
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: f058b9ff933d39ab010000006920bdf99e09563045eb333e (good)
;; QUESTION SECTION:
;dev1.dept.example.lab.         IN      A

;; AUTHORITY SECTION:
dept.example.lab.       86400   IN      NS      ns3.dept.example.lab.

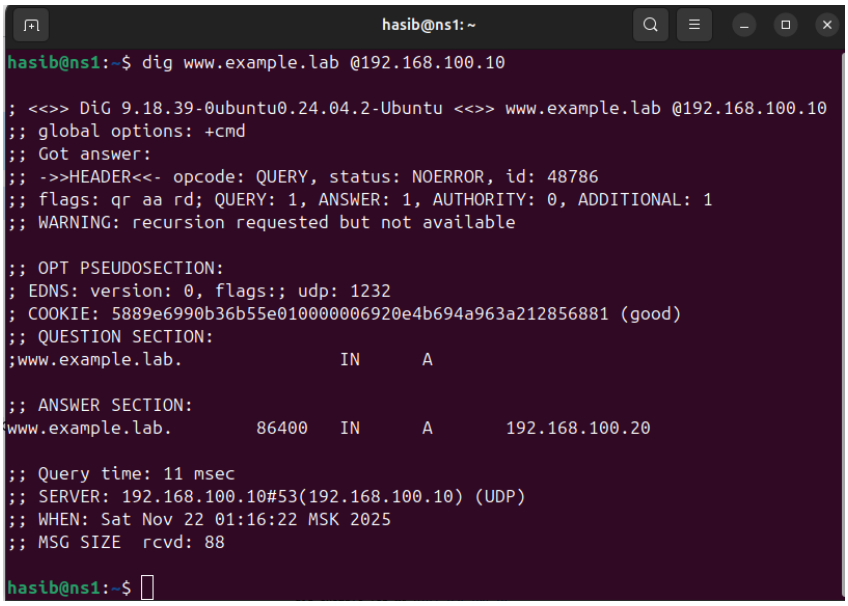
;; Query time: 1 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Fri Nov 21 22:31:05 MSK 2025
;; MSG SIZE  rcvd: 96
```

ns1 has learned that dev1.dept.example.lab is a subdomain of dept.example.lab, and it has stored ns3.dept.example.lab as the NS record for this subdomain. The resolver has used the glue A record to get the IP address of that NS and is then able to send the query directly to ns3. The A record is returned authoritatively by ns3. This means that the entire real-life resolution flow of DNS delegation has been successful.

TASK 5 - Testing and Verification

This task tests three DNS servers. These are essential for verifying the correctness of querying, zone resolution, delegation, and failover of the BIND DNS server [1], [11].

Query the primary server:



The above is in the present perfect tense, indicating that the resolution has already been completed. ns1 is configured as the authoritative master DNS for the example.lab domain, and its forward zone file db.example.lab has an A record for www.example.lab (192.168.100.20) stored [1]. As a result, when a query is sent, ns1 returns the stored A record as the authoritative answer.

Query specific record types

```
hasib@ns1:~$ dig example.lab MX @192.168.100.10

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> example.lab MX @192.168.100.10
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 9254
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 2
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: ec4b277b85e5e315010000006920e4e8bbd5d5507572e13b (good)
;; QUESTION SECTION:
;example.lab.                   IN      MX

;; ANSWER SECTION:
example.lab.            86400   IN      MX      10 mail.example.lab.

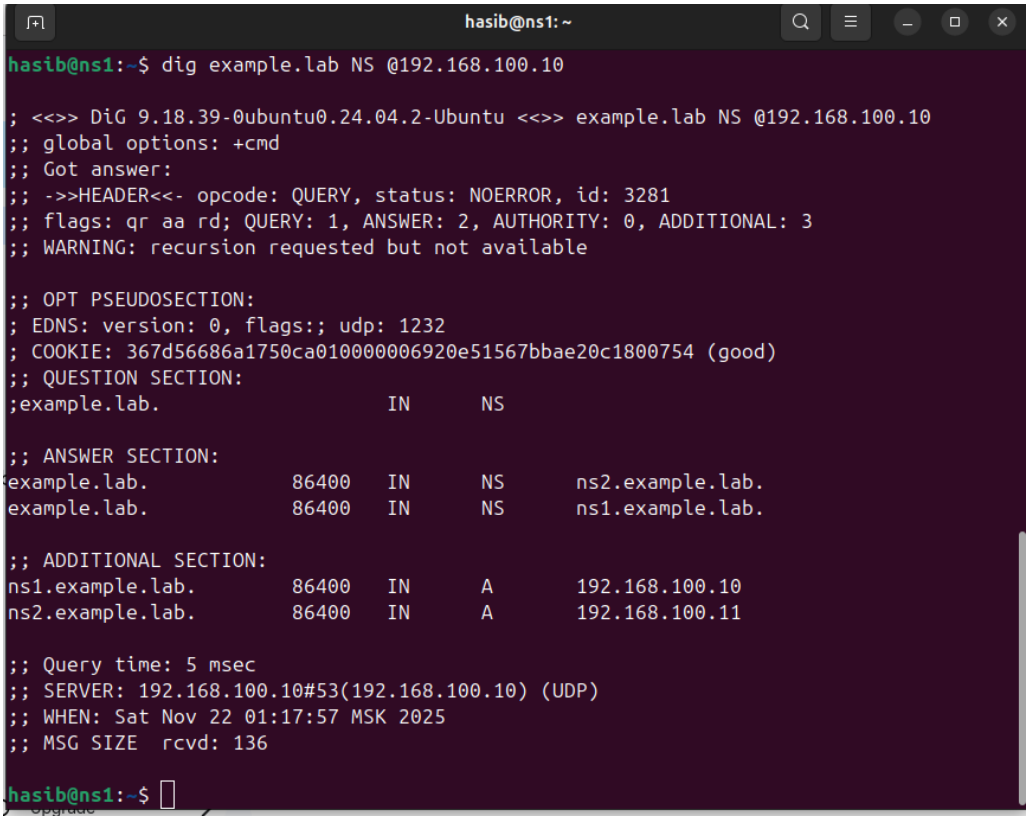
;; ADDITIONAL SECTION:
mail.example.lab.       86400   IN      A       192.168.100.25

;; Query time: 2 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 01:17:12 MSK 2025
;; MSG SIZE  rcvd: 105

hasib@ns1:~$
```

This indicates that MX resolution has been done correctly. The MX record is used to point to the domain's mail server, and the forward zone file already has the MX 10 mail.example.lab value configured. So, when a mail query is sent, ns1 returns the specified MX record as the authoritative answer [1].

NS record test



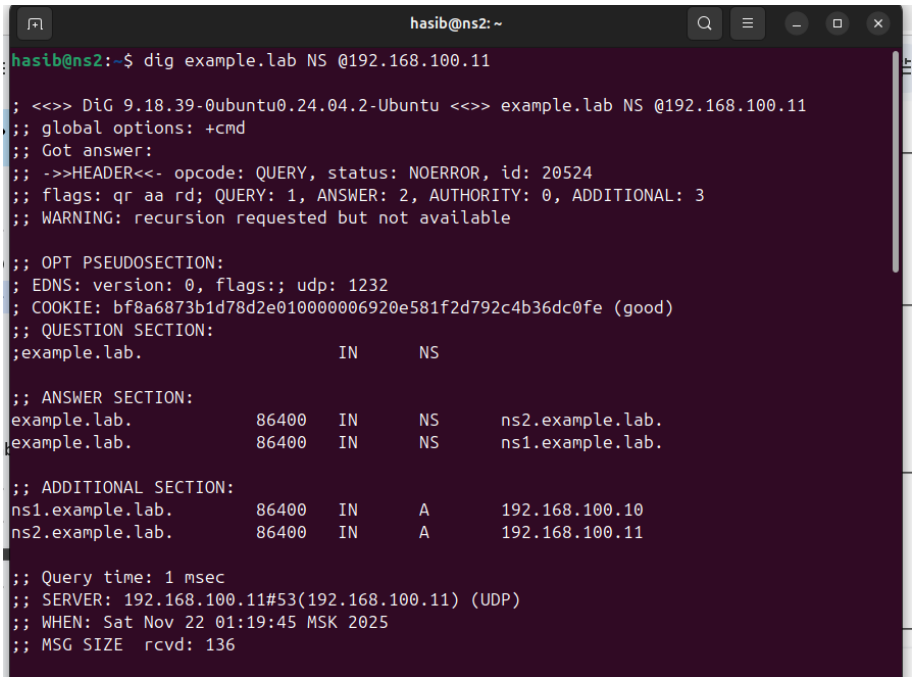
ns1 and ns2 are designated as authoritative DNS servers for the example.lab domain, and these two NS records are added to the forward zone file. As a result, the authoritative responsibility for the domain is distributed between ns1 and ns2, ensuring redundancy and reliability [1], [11].

Reverse lookup (PTR) (from ns1)



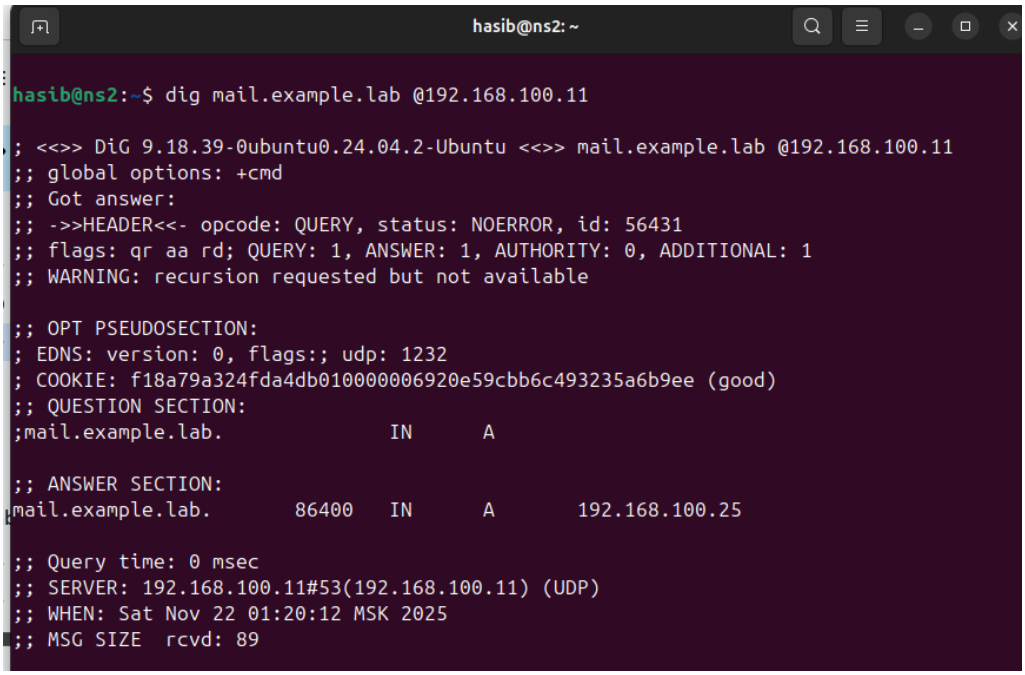
The reverse zone file (db.192.168.100) has 20 PTR www.example.lab entries set, so when a PTR lookup is performed, ns1 returns that PTR record as the authoritative answer.

NS record (from ns2)



ns2 is serving the same NS record as a slave.

A record (from ns2)



The A record for mail.example.lab has been transferred to the slave zone file.

Reverse lookup PTR (from ns2)

```
hasib@ns2:~$ dig -x 192.168.100.35 @192.168.100.11

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> -x 192.168.100.35 @192.168.100.11
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 16397
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: e8a227356fedc39e010000006920e5b1d3617bf32421e28d (good)
;; QUESTION SECTION:
;35.100.168.192.in-addr.arpa.   IN      PTR

;; ANSWER SECTION:
35.100.168.192.in-addr.arpa. 86400 IN   PTR     db.example.lab.

;; Query time: 0 msec
;; SERVER: 192.168.100.11#53(192.168.100.11) (UDP)
;; WHEN: Sat Nov 22 01:20:33 MSK 2025
;; MSG SIZE  rcvd: 112
```

Zone transfer to db.example.lab on PTR 35.

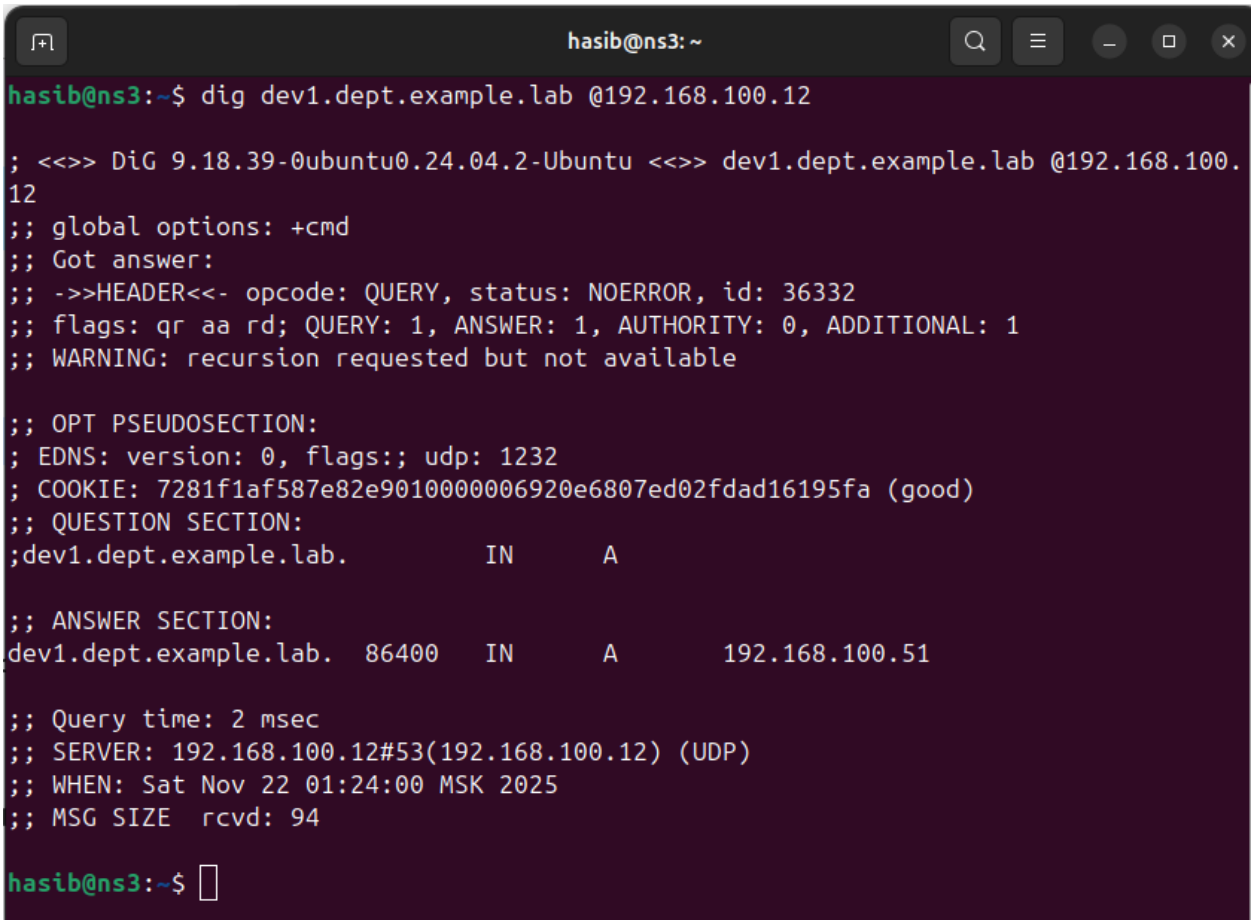
Check if Zone Transfer file exists (from ns2)

```
hasib@ns2:~$ ls -la /var/cache/bind/
total 24
drwxrwxr-x  2 root bind 4096 Nov 21 21:18 .
drwxr-xr-x 22 root root 4096 Nov 20 22:04 ..
-rw-r--r--  1 bind bind  688 Nov 21 20:56 db.192.168.100
-rw-r--r--  1 bind bind  717 Nov 21 20:56 db.example.lab
-rw-r--r--  1 bind bind  221 Nov 21 18:04 managed-keys.bind
-rw-r--r--  1 bind bind 1210 Nov 21 18:03 managed-keys.bind.jnl
hasib@ns2:~$
```

The slave server did not edit the zone file itself, BIND created this file itself [1].

Delegated DNS test (ns3)

dev1 resolve test (from ns3)

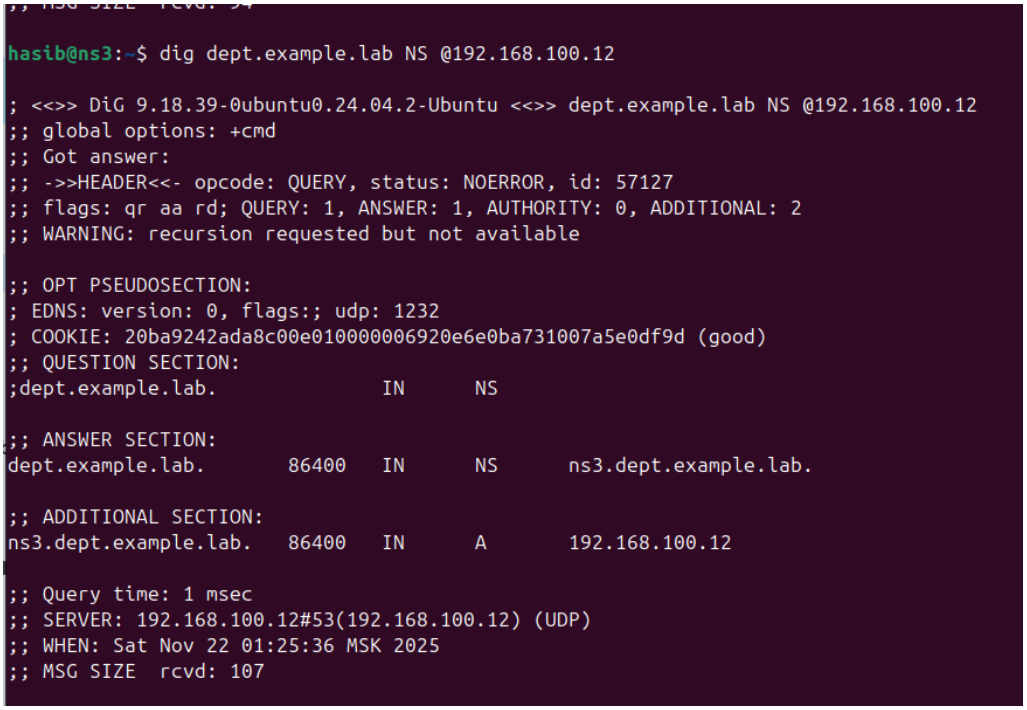


dev2 resolve test (from ns3)



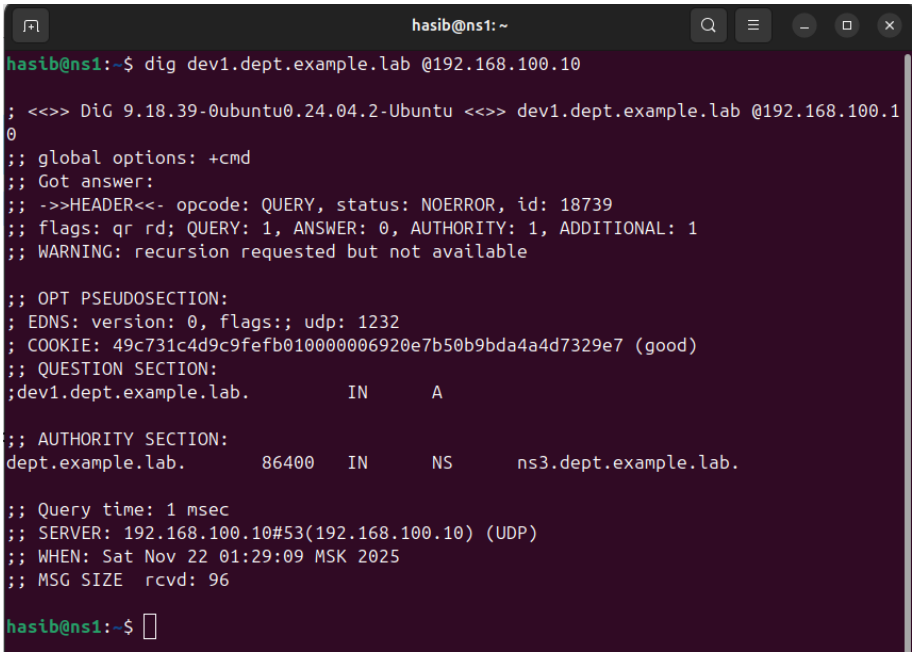
There is an A record in the forward zone file (db.dept.example.lab) of ns3.

Delegated NS record (from ns3)



Since ns3 is authoritative, it serves the subdomain's NS record itself.

Delegation full path



ns1 has already identified dev1.dept.example.lab as a subdomain, and has obtained ns3.dept.example.lab as the delegation target from the NS record in the parent zone. Using the

Glue A record, ns1 determines the IP address of ns3 as 192.168.100.12, and then sends the query to ns3's DNS server. ns3 authoritatively returns the correct answer, and ns1 returns that answer to the user. The entire chain has worked—this is clear evidence of delegation success [11], [14].

Shutting down ns1

```
hasib@ns1:~$ sudo systemctl stop bind9
```

Master server DNS is down to test slave behavior.

See if ns2 responds

```
hasib@ns2:~$ dig www.example.lab @192.168.100.11

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> www.example.lab @192.168.100.11
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 3536
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: a53622c486e73241010000006920e8b2cdff57961c7c10b0 (good)
;; QUESTION SECTION:
;www.example.lab.               IN      A

;; ANSWER SECTION:
www.example.lab.        86400   IN      A       192.168.100.20

;; Query time: 3 msec
;; SERVER: 192.168.100.11#53(192.168.100.11) (UDP)
;; WHEN: Sat Nov 22 01:33:22 MSK 2025
;; MSG SIZE  rcvd: 88

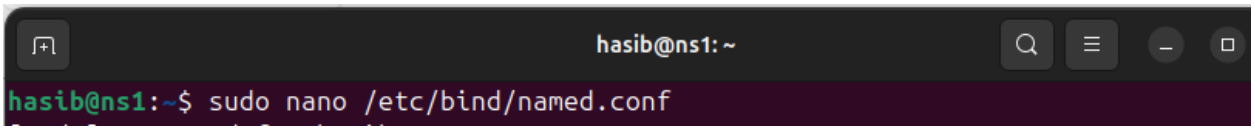
hasib@ns2:~$
```

Even though ns1 is down, ns2 uses slave zone data to give an authoritative answer. This means the zone transfer was successful.

TASK 6 - Advanced Configuration Tasks

Task 6.1: Configure DNS Logging

Server: Open the config file on ns1



/etc/bind/named.conf is used as the main configuration file for BIND 9, and other necessary configuration files are included from here [1], [16]. Since it is convenient to redirect logs correctly from the time the server starts, logging statements are also placed in this main named.conf, so that log management is effective from the initialization phase of BIND [17], [18].

Adding a logging section:



logging { ... }; The entire BIND server logging setup is defined inside the block, where two channels (query_log, transfer_log) and three categories (queries, xfer-in, xfer-out) are mapped appropriately [1], [17]. Among them, the channel query_log is specifically created for collecting DNS query-related logs, so that detailed information about each DNS query coming to the server is recorded sequentially.

channel query_log { ... }; A logging channel named “query_log” is created through the block, where where and how the log will go according to the BIND channel concept - file, syslog or console - has been determined [17]. Here the /var/log/named/query.log file is set as the log destination and a total of three rotated files (query.log, query.log.0, query.log.1, query.log.2) are

kept with versions 3, while rotation is triggered when each file is 5 MB with size 5m. severity info; It is possible to record general query-related informational logs with . In addition, print-time yes; adds a timestamp to the beginning of each log line, print-severity yes; displays the log severity in text, and print-category yes; clearly shows which category (queries, resolver, security, etc.) the log came from [1], [17], [19]. In category mapping, category queries { query_log; }; sends all DNS query-related logs to the query_log channel, so that the information for each incoming query is stored in a separate file. Similarly, category xfer-in { transfer_log; }; sends the incoming zone transfer logs—where the slave server receives data from the master—to the transfer_log, and category xfer-out { transfer_log; }; routes the outgoing zone transfer logs to the same channel. In this way, query and zone-transfer logs are kept in separate files in a well-organized manner [1], [20].

Creating log directory

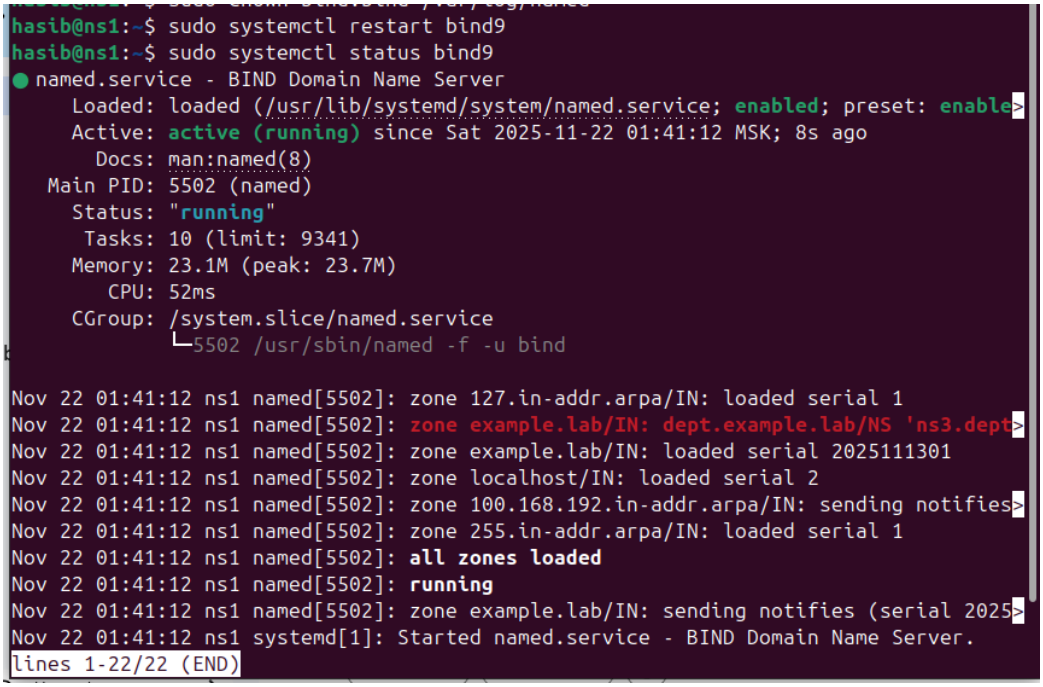
```
hasib@ns1:~$ sudo mkdir -p /var/log/named
```

```
hasib@ns1:~$ sudo chown bind:bind /var/log/named
```

The /var/log/named directory was created by running mkdir -p /var/log/named, because we used this path in the logging block and without the directory the named service would not be able to create any log files. Then the ownership of the directory was set to the bind user and group by running chown bind:bind /var/log/named, since the BIND service in Ubuntu/Debian runs as the bind user [21]. So, setting this permission ensures that the service has the necessary permissions to write to the log file.

Bind restart

```
hasib@ns1:~$ sudo systemctl restart bind9
```



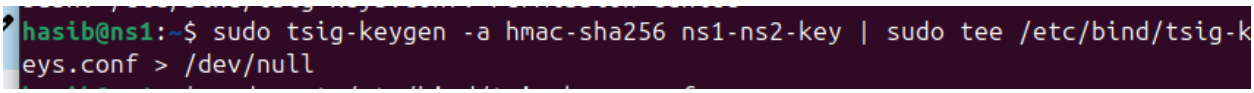
Expected output was active (running).

Task 6.2: Implement TSIG for Secure Zone Transfers

The zone transfer (AXFR/IXFR) between ns1 and ns2 is signed with the TSIG key.

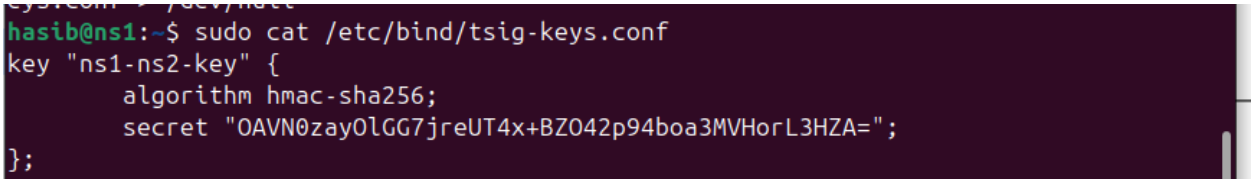
TSIG (Transaction SIGnature) is a shared secret key-based authentication used in DNS server-to-server communication (zone transfer, notify, dynamic update) [1], [23].

Generate HMAC-SHA256 key



A TSIG key for BIND is generated using tsig-keygen, where the modern and secure HMAC-SHA256 algorithm is selected with the -a hmac-sha256 option. The key name is ns1-ns2-key, which is then used as a reference in named.conf [24], [25]. The output of the command is written directly to the /etc/bind/tsig-keys.conf file, so that the key is securely stored in a separate config file. The generated TSIG key typically contains a 256-bit secret, which has been proven to be strong enough according to security guidelines [25].

View Key File



The key "ns1-ns2-key" { ... }; block has been added to named.conf, which provides the exact syntax configuration required for TSIG. Here, the HMAC algorithm used for signing is specified, using algorithm hmac-sha256; and the secret "BASE64_KEY==" is set to the Base64–encoded shared secret. The same secret is used on both servers, ensuring authenticated, secure communication between ns1 and ns2.

Including the key in named.conf

```
hasib@ns1:~$ sudo nano /etc/bind/named.conf
hasib@ns1:~$
```

Adding:

```
include "/etc/bind/tsig-keys.conf";
```

The include statement merges the config of a separate file into the main config [1], [16]. As a result, the key block is parsed as part of named.conf.

Modify the zone definition to use TSIG

```
hasib@ns2:~$ sudo nano /etc/bind/named.conf.local
hasib@ns2:~$
```

Updating the forward zone

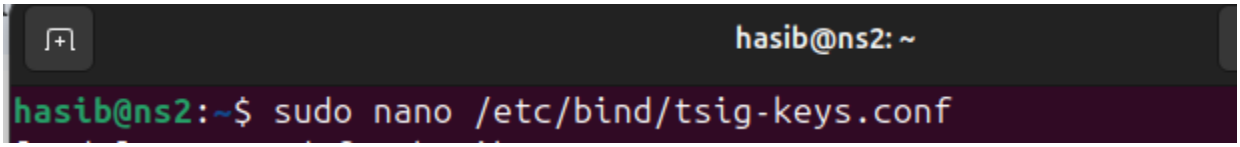
```
zone "example.lab" {
    type master;
    file "/etc/bind/zones/db.example.lab";

    allow-transfer { key ns1-ns2-key; };

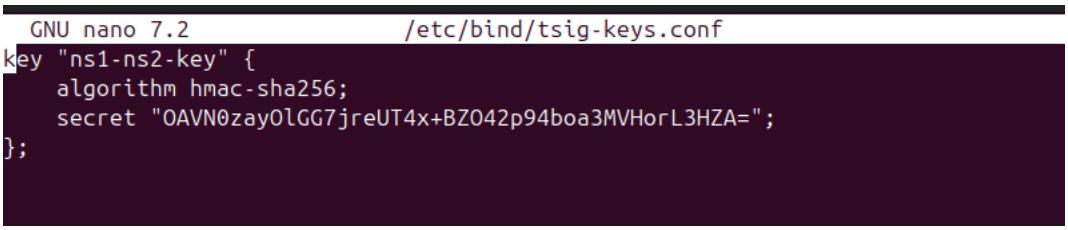
    notify yes;
};
```

Previously, only IP–based ACLs were set using allow-transfer { 192.168.100.11; }; , which depended only on the IP address. Now, the configuration has been updated to use allow-transfer { key ns1-ns2-key; }; , so that zone transfers are only allowed to the owner of that TSIG key (i.e., authenticated ns2). The transfer is only allowed when ns2 sends a request signed with the same shared secret. The previous system was relatively weak due to the risk of IP spoofing, but using TSIG makes the request rejected if the shared secret mismatches, thus significantly strengthening security [22], [23].

Copying TSIG key to ns2

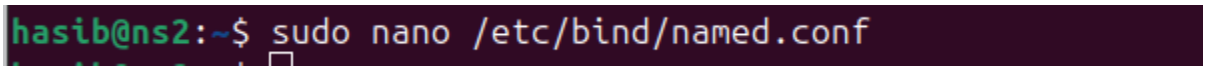


updating configuration

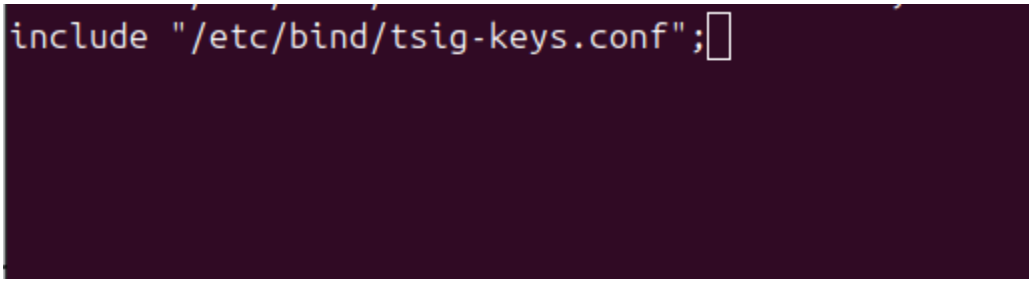


The secret in the key file of ns1 is copied exactly the same in ns2, because it has been working as a shared secret in TSIG. If this secret has a mismatch of even one letter on both sides, then TSIG validation has definitely failed, which is a mandatory rule of the protocol.

Including TSIG in ns2



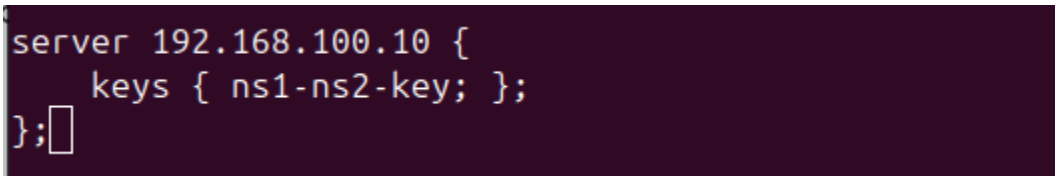
Adding:

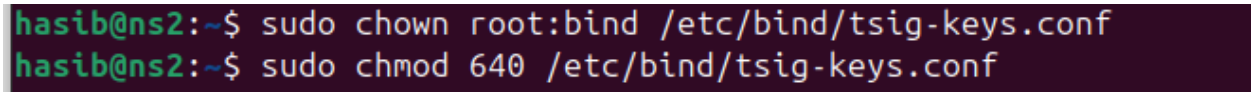


ns2 also has a key defined, so that BIND knows the ns1-ns2-key key thing.

Adding server block at ns2

This is added to /etc/bind/named.conf:





```
hasib@ns2:~$ ls -l /etc/bind/tsig-keys.conf
-rw-r----- 1 root bind 109 Nov 22 02:48 /etc/bind/tsig-keys.conf
```

The server 192.168.100.10 { ... } block is used to inform BIND that server-to-server requests such as zone transfers or notify sent to ns1 (192.168.100.10) are handled according to a specific configuration [1], [16]. Here, keys { ns1-ns2-key; }; are added to ensure that all requests sent to ns1 by ns2 are TSIG-signed. ns2 now authenticates using ns1-ns2-key when communicating with ns1 [22], [23]. A key-based rule has been added to the allow-transfer in ns1, and the same key is used in the server block in ns2. As a result, TSIG-based trust and secure communication "friendship" have been established on both sides**.**

Restarting & Testing

```
hasib@ns2:~$ sudo systemctl restart bind9
```

```
hasib@ns2:~$ sudo systemctl status bind9
● named.service - BIND Domain Name Server
     Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: en>
     Active: active (running) since Sat 2025-11-22 02:49:25 MSK; 8s ago
       Docs: man:named(8)
   Main PID: 6797 (named)
     Status: "running"
      Tasks: 10 (limit: 9341)
     Memory: 23.1M (peak: 23.4M)
        CPU: 47ms
     CGroup: /system.slice/named.service
             └─6797 /usr/sbin/named -f -u bind

Nov 22 02:49:25 ns2 named[6797]: zone 0.in-addr.arpa/IN: loaded serial 1
Nov 22 02:49:25 ns2 named[6797]: zone 255.in-addr.arpa/IN: loaded serial 1
Nov 22 02:49:25 ns2 named[6797]: zone 100.168.192.in-addr.arpa/IN: loaded seria>
Nov 22 02:49:25 ns2 named[6797]: zone example.lab/IN: loaded serial 2025111301
Nov 22 02:49:25 ns2 named[6797]: zone localhost/IN: loaded serial 2
Nov 22 02:49:25 ns2 named[6797]: all zones loaded
Nov 22 02:49:25 ns2 named[6797]: running
Nov 22 02:49:25 ns2 named[6797]: zone 100.168.192.in-addr.arpa/IN: sending noti>
Nov 22 02:49:25 ns2 systemd[1]: Started named.service - BIND Domain Name Server.
Nov 22 02:49:25 ns2 named[6797]: zone example.lab/IN: sending notifies (serial >
```

Secure Zone Transfer Test (AXFR)

Usually done with TSH test dig.

```
hasib@ns2:~$ dig @192.168.100.10 example.lab AXFR -k /etc/bind/tsig-keys.conf

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 example.lab AXFR -k /etc/bind/tsig-keys.conf
; (1 server found)
;; global options: +cmd
example.lab.            86400   IN      SOA     ns1.example.lab. admin.example.lab. 2025111301 3600 1800 604800 86400
example.lab.            86400   IN      NS      ns1.example.lab.
example.lab.            86400   IN      NS      ns2.example.lab.
example.lab.            86400   IN      MX      10 mail.example.lab.
db.example.lab.         86400   IN      A       192.168.100.35
dept.example.lab.       86400   IN      NS      ns3.dept.example.lab.
ftp.example.lab.        86400   IN      A       192.168.100.30
mail.example.lab.       86400   IN      A       192.168.100.25
ns1.example.lab.        86400   IN      A       192.168.100.10
ns2.example.lab.        86400   IN      A       192.168.100.11
ns3.example.lab.        86400   IN      A       192.168.100.12
smtp.example.lab.       86400   IN      CNAME   mail.example.lab.
webmail.example.lab.    86400   IN      CNAME   www.example.lab.
www.example.lab.        86400   IN      A       192.168.100.20
example.lab.            86400   IN      SOA     ns1.example.lab. admin.example.lab. 2025111301 3600 1800 604800 86400
ns1-ns2-key.            0       ANY     TSIG    hmac-sha256. 1763769422 300 32 bEDtDv4HJ8Tv26znpQuhbjwm3/t4Dqk2IiJRgI2Eb
cA= 39866 NOERROR 0
;; Query time: 0 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (TCP)
;; WHEN: Sat Nov 22 02:57:02 MSK 2025
;; XFR size: 15 records (messages 1, bytes 478)
```

dig @192.168.100.11 example.lab A request for a full zone transfer from ns2 to example.lab was sent using AXFR, and the -k /etc/bind/tsig-keys.conf option was used to instruct dig to apply the TSIG key, which depends on the BIND/dig version [1], [19]. With the configuration set correctly, ns1 verified the TSIG signature of the request, then AXFR approved the transfer, and the dig output successfully displayed the full zone-SOA, NS, A, MX, etc. with all records.

TASK 7 - Troubleshooting and Debugging

The purpose of Task 7 is to identify, debug, and fix DNS problems.

Troubleshooting Steps:

Issue 1: BIND Service Fails to Start

1. Configuration Syntax Check

```
hasib@ns1:~$ sudo named-checkconf
```

```
hasib@ns2:~$ sudo named-checkconf
```

```
hasib@ns3:~$ sudo named-checkconf
```

The entire BIND configuration was parsed and the syntax was checked by running named-checkconf [1]. This tool checked /etc/bind/named.conf along with all its included files such as named.conf.options, named.conf.local. It also used the ability to show errors with line-numbers if there is a missing semicolon, duplicate block or wrong option. As a result, the syntax-check of the entire configuration was successfully completed at this stage.

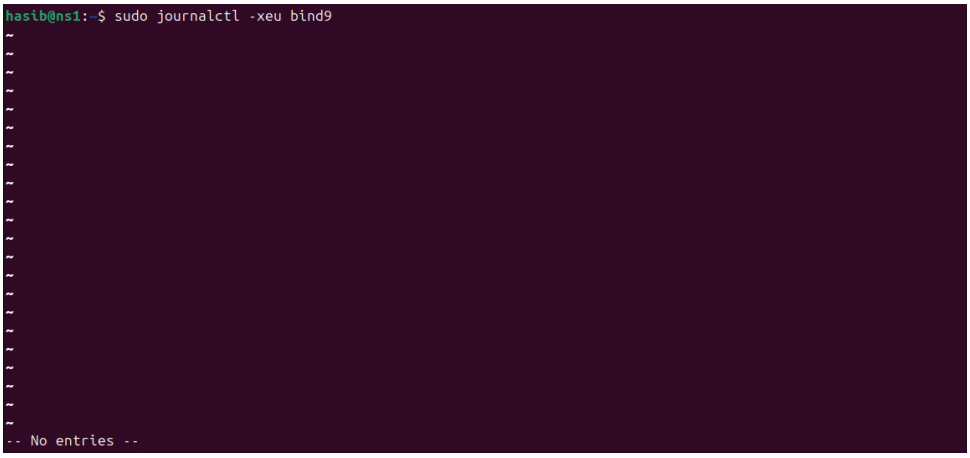
2. Zone File Syntax Check

```
hasib@ns1:~$ sudo named-checkzone example.lab /etc/bind/zones/db.example.lab
zone example.lab/IN: dept.example.lab/NS 'ns3.dept.example.lab' has no REQUIRED GLUE address records (A or AAAA)
zone example.lab/IN: dept.example.lab/NS 'ns3.dept.example.lab' (out of zone) has no addresses records (A or AAAA)
zone example.lab/IN: loaded serial 2025111301
OK
```

```
hasib@ns3:~$ sudo named-checkzone dept.example.lab /etc/bind/zones/db.dept.example.lab
zone dept.example.lab/IN: loaded serial 2025111301
OK
```

The named-checkzone <zone> <file> command checks whether the specified zone file is valid and whether its SOA, NS, and other records are formatted correctly [1]. In this case, the zone name is example.lab and the file path is /etc/bind/zones/db.example.lab, and the tool verifies that the entire zone structure is loaded correctly.

3. Check System Logs







In this step, we have used journalctl -xeu bind9 to view the latest logs of the bind9 service from the systemd journal, where -x shows additional explanations, -e takes the command directly to the end of the log, and the -u bind9 option filters out logs related to the bind9 service. This way, bind9 runtime errors, warnings, start/stop events, etc. can be easily found [29], [30].

4. Check Port Conflicts

```
hasib@ns1:~$ sudo ss -tulpn | grep :53
udp   UNCONN 0      0                          192.168.100.10:53         0.0.0.0:*    users:(("named",pid=1240,fd=63))

udp   UNCONN 0      0                          192.168.100.10:53         0.0.0.0:*    users:(("named",pid=1240,fd=62))

udp   UNCONN 0      0                          192.168.100.10:53         0.0.0.0:*    users:(("named",pid=1240,fd=61))

udp   UNCONN 0      0                          192.168.100.10:53         0.0.0.0:*    users:(("named",pid=1240,fd=60))

udp   UNCONN 0      0                               127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1240,fd=51))

udp   UNCONN 0      0                               127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1240,fd=50))

udp   UNCONN 0      0                               127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1240,fd=49))

udp   UNCONN 0      0                               127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1240,fd=48))

udp   UNCONN 0      0                              127.0.0.54:53         0.0.0.0:*    users:(("systemd-resolve",pid=604,f
d=16))
udp   UNCONN 0      0                           127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=604,f
d=14))
udp   UNCONN 0      0                                 0.0.0.0:5353       0.0.0.0:*    users:(("avahi-daemon",pid=902,fd=1
2))
udp   UNCONN 0      0                                   [::1]:53            [::]:*    users:(("named",pid=1240,fd=68))

udp   UNCONN 0      0                                   [::1]:53            [::]:*    users:(("named",pid=1240,fd=69))

udp   UNCONN 0      0                                   [::1]:53            [::]:*    users:(("named",pid=1240,fd=70))
```

```
udp   UNCONN 0      0                                   [::1]:53            [::]:*    users:(("named",pid=1240,fd=71))

udp   UNCONN 0      0       [fe80::20c:29ff:feae:b1fd]%ens33:53            [::]:*    users:(("named",pid=1240,fd=76))

udp   UNCONN 0      0       [fe80::20c:29ff:feae:b1fd]%ens33:53            [::]:*    users:(("named",pid=1240,fd=77))

udp   UNCONN 0      0       [fe80::20c:29ff:feae:b1fd]%ens33:53            [::]:*    users:(("named",pid=1240,fd=78))

udp   UNCONN 0      0       [fe80::20c:29ff:feae:b1fd]%ens33:53            [::]:*    users:(("named",pid=1240,fd=79))

udp   UNCONN 0      0      [fe80::3a55:df58:97c6:86c0]%ens34:53            [::]:*    users:(("named",pid=1240,fd=84))

udp   UNCONN 0      0      [fe80::3a55:df58:97c6:86c0]%ens34:53            [::]:*    users:(("named",pid=1240,fd=85))

udp   UNCONN 0      0      [fe80::3a55:df58:97c6:86c0]%ens34:53            [::]:*    users:(("named",pid=1240,fd=86))

udp   UNCONN 0      0      [fe80::3a55:df58:97c6:86c0]%ens34:53            [::]:*    users:(("named",pid=1240,fd=87))

udp   UNCONN 0      0                                    [::]:5353          [::]:*    users:(("avahi-daemon",pid=902,fd=1
3))
tcp   LISTEN 0      10                        192.168.100.10:53         0.0.0.0:*    users:(("named",pid=1240,fd=67))

tcp   LISTEN 0      10                        192.168.100.10:53         0.0.0.0:*    users:(("named",pid=1240,fd=66))

tcp   LISTEN 0      10                        192.168.100.10:53         0.0.0.0:*    users:(("named",pid=1240,fd=65))

tcp   LISTEN 0      10                        192.168.100.10:53         0.0.0.0:*    users:(("named",pid=1240,fd=64))
```

```
tcp   LISTEN 0      10                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1240,fd=54))

tcp   LISTEN 0      10                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1240,fd=52))

tcp   LISTEN 0      4096                        127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=604,f
d=15))
tcp   LISTEN 0      10                                  [::1]:53            [::]:*    users:(("named",pid=1240,fd=73))

tcp   LISTEN 0      10                                  [::1]:53            [::]:*    users:(("named",pid=1240,fd=74))

tcp   LISTEN 0      10                                  [::1]:53            [::]:*    users:(("named",pid=1240,fd=75))

tcp   LISTEN 0      10                                  [::1]:53            [::]:*    users:(("named",pid=1240,fd=72))

tcp   LISTEN 0      10      [fe80::20c:29ff:feae:b1fd]%ens33:53            [::]:*    users:(("named",pid=1240,fd=80))

tcp   LISTEN 0      10      [fe80::20c:29ff:feae:b1fd]%ens33:53            [::]:*    users:(("named",pid=1240,fd=81))

tcp   LISTEN 0      10      [fe80::20c:29ff:feae:b1fd]%ens33:53            [::]:*    users:(("named",pid=1240,fd=82))

tcp   LISTEN 0      10      [fe80::20c:29ff:feae:b1fd]%ens33:53            [::]:*    users:(("named",pid=1240,fd=83))

tcp   LISTEN 0      10     [fe80::3a55:df58:97c6:86c0]%ens34:53            [::]:*    users:(("named",pid=1240,fd=88))

tcp   LISTEN 0      10     [fe80::3a55:df58:97c6:86c0]%ens34:53            [::]:*    users:(("named",pid=1240,fd=89))

tcp   LISTEN 0      10     [fe80::3a55:df58:97c6:86c0]%ens34:53            [::]:*    users:(("named",pid=1240,fd=90))

tcp   LISTEN 0      10     [fe80::3a55:df58:97c6:86c0]%ens34:53            [::]:*    users:(("named",pid=1240,fd=91))
```

```
hasib@ns2:~$ sudo ss -tulpn | grep :53
udp   UNCONN 0      0                                  0.0.0.0:5353       0.0.0.0:*    users:(("avahi-daemon",pid=903,fd=12))
udp   UNCONN 0      0                        192.168.100.11:53         0.0.0.0:*    users:(("named",pid=1234,fd=63))
udp   UNCONN 0      0                        192.168.100.11:53         0.0.0.0:*    users:(("named",pid=1234,fd=62))
udp   UNCONN 0      0                        192.168.100.11:53         0.0.0.0:*    users:(("named",pid=1234,fd=61))
udp   UNCONN 0      0                        192.168.100.11:53         0.0.0.0:*    users:(("named",pid=1234,fd=60))
udp   UNCONN 0      0                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1234,fd=51))
udp   UNCONN 0      0                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1234,fd=50))
udp   UNCONN 0      0                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1234,fd=49))
udp   UNCONN 0      0                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1234,fd=48))
udp   UNCONN 0      0                            127.0.0.54:53         0.0.0.0:*    users:(("systemd-resolve",pid=614,fd=16))
udp   UNCONN 0      0                         127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=614,fd=14))
udp   UNCONN 0      0                                    [::]:5353          [::]:*    users:(("avahi-daemon",pid=903,fd=13))
udp   UNCONN 0      0                                   [::1]:53            [::]:*    users:(("named",pid=1234,fd=68))
udp   UNCONN 0      0                                   [::1]:53            [::]:*    users:(("named",pid=1234,fd=69))
udp   UNCONN 0      0                                   [::1]:53            [::]:*    users:(("named",pid=1234,fd=70))
```

```
udp   UNCONN 0      0                                   [::1]:53            [::]:*    users:(("named",pid=1234,fd=71))
udp   UNCONN 0      0       [fe80::20c:29ff:fe0f:8cee]%ens33:53            [::]:*    users:(("named",pid=1234,fd=76))
udp   UNCONN 0      0       [fe80::20c:29ff:fe0f:8cee]%ens33:53            [::]:*    users:(("named",pid=1234,fd=77))
udp   UNCONN 0      0       [fe80::20c:29ff:fe0f:8cee]%ens33:53            [::]:*    users:(("named",pid=1234,fd=78))
udp   UNCONN 0      0       [fe80::20c:29ff:fe0f:8cee]%ens33:53            [::]:*    users:(("named",pid=1234,fd=79))
udp   UNCONN 0      0      [fe80::c3ba:dabb:3674:380a]%ens34:53            [::]:*    users:(("named",pid=1234,fd=84))
udp   UNCONN 0      0      [fe80::c3ba:dabb:3674:380a]%ens34:53            [::]:*    users:(("named",pid=1234,fd=85))
udp   UNCONN 0      0      [fe80::c3ba:dabb:3674:380a]%ens34:53            [::]:*    users:(("named",pid=1234,fd=86))
udp   UNCONN 0      0      [fe80::c3ba:dabb:3674:380a]%ens34:53            [::]:*    users:(("named",pid=1234,fd=87))
tcp   LISTEN 0      10                       192.168.100.11:53          0.0.0.0:*    users:(("named",pid=1234,fd=67))
tcp   LISTEN 0      10                       192.168.100.11:53          0.0.0.0:*    users:(("named",pid=1234,fd=66))
tcp   LISTEN 0      10                       192.168.100.11:53          0.0.0.0:*    users:(("named",pid=1234,fd=65))
tcp   LISTEN 0      10                       192.168.100.11:53          0.0.0.0:*    users:(("named",pid=1234,fd=64))
tcp   LISTEN 0      10                            127.0.0.1:53          0.0.0.0:*    users:(("named",pid=1234,fd=58))
tcp   LISTEN 0      10                            127.0.0.1:53          0.0.0.0:*    users:(("named",pid=1234,fd=56))
tcp   LISTEN 0      10                            127.0.0.1:53          0.0.0.0:*    users:(("named",pid=1234,fd=54))
```

```
tcp   LISTEN 0      10                            127.0.0.1:53          0.0.0.0:*    users:(("named",pid=1234,fd=52))
tcp   LISTEN 0      4096                         127.0.0.54:53          0.0.0.0:*    users:(("systemd-resolve",pid=614,fd=17))
tcp   LISTEN 0      4096                      127.0.0.53%lo:53          0.0.0.0:*    users:(("systemd-resolve",pid=614,fd=15))
tcp   LISTEN 0      10                                  [::1]:53            [::]:*    users:(("named",pid=1234,fd=72))
tcp   LISTEN 0      10                                  [::1]:53            [::]:*    users:(("named",pid=1234,fd=73))
tcp   LISTEN 0      10                                  [::1]:53            [::]:*    users:(("named",pid=1234,fd=74))
tcp   LISTEN 0      10                                  [::1]:53            [::]:*    users:(("named",pid=1234,fd=75))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe0f:8cee]%ens33:53            [::]:*    users:(("named",pid=1234,fd=80))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe0f:8cee]%ens33:53            [::]:*    users:(("named",pid=1234,fd=81))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe0f:8cee]%ens33:53            [::]:*    users:(("named",pid=1234,fd=82))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe0f:8cee]%ens33:53            [::]:*    users:(("named",pid=1234,fd=83))
tcp   LISTEN 0      10     [fe80::c3ba:dabb:3674:380a]%ens34:53            [::]:*    users:(("named",pid=1234,fd=88))
tcp   LISTEN 0      10     [fe80::c3ba:dabb:3674:380a]%ens34:53            [::]:*    users:(("named",pid=1234,fd=89))
tcp   LISTEN 0      10     [fe80::c3ba:dabb:3674:380a]%ens34:53            [::]:*    users:(("named",pid=1234,fd=90))
tcp   LISTEN 0      10     [fe80::c3ba:dabb:3674:380a]%ens34:53            [::]:*    users:(("named",pid=1234,fd=91))
```

```
hasib@ns3:~$ sudo ss -tulpn | grep :53
udp   UNCONN 0      0                                0.0.0.0:5353       0.0.0.0:*    users:(("avahi-daemon",pid=938,fd=12
))
udp   UNCONN 0      0                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=92))

udp   UNCONN 0      0                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=94))

udp   UNCONN 0      0                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=93))

udp   UNCONN 0      0                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=95))

udp   UNCONN 0      0                           127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=51))

udp   UNCONN 0      0                           127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=50))

udp   UNCONN 0      0                           127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=49))

udp   UNCONN 0      0                           127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=48))

udp   UNCONN 0      0                          127.0.0.54:53         0.0.0.0:*    users:(("systemd-resolve",pid=606,fd
=16))
udp   UNCONN 0      0                       127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=606,fd
=14))
udp   UNCONN 0      0                                [::]:5353          [::]:*    users:(("avahi-daemon",pid=938,fd=13
))
udp   UNCONN 0      0                               [::1]:53            [::]:*    users:(("named",pid=1237,fd=60))

udp   UNCONN 0      0                               [::1]:53            [::]:*    users:(("named",pid=1237,fd=61))
```

```
tcp   LISTEN 0      10                           127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=52))

tcp   LISTEN 0      4096                    127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=606,fd
=15))
tcp   LISTEN 0      4096                       127.0.0.54:53         0.0.0.0:*    users:(("systemd-resolve",pid=606,fd
=17))
tcp   LISTEN 0      10                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=97))

tcp   LISTEN 0      10                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=98))

tcp   LISTEN 0      10                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=99))

tcp   LISTEN 0      10                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=96))

tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=73))

tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=72))

tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=74))

tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=75))

tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=89))

tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=91))

tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=90))

tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=88))
```

```
tcp   LISTEN 0      10                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=99))

tcp   LISTEN 0      10                      192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=96))

tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=73))

tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=72))

tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=74))

tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=75))

tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=89))

tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=91))

tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=90))

tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=88))

tcp   LISTEN 0      10                              [::1]:53            [::]:*    users:(("named",pid=1237,fd=67))

tcp   LISTEN 0      10                              [::1]:53            [::]:*    users:(("named",pid=1237,fd=66))

tcp   LISTEN 0      10                              [::1]:53            [::]:*    users:(("named",pid=1237,fd=64))

tcp   LISTEN 0      10                              [::1]:53            [::]:*    users:(("named",pid=1237,fd=65))
```

```
udp   UNCONN 0      0                                [::1]:53           [::]:*    users:(("named",pid=1237,fd=61))
udp   UNCONN 0      0                                [::1]:53           [::]:*    users:(("named",pid=1237,fd=62))
udp   UNCONN 0      0                                [::1]:53           [::]:*    users:(("named",pid=1237,fd=63))
udp   UNCONN 0      0       [fe80::20c:29ff:fe8c:2a1]%ens33:53           [::]:*    users:(("named",pid=1237,fd=68))
udp   UNCONN 0      0       [fe80::20c:29ff:fe8c:2a1]%ens33:53           [::]:*    users:(("named",pid=1237,fd=71))
udp   UNCONN 0      0       [fe80::20c:29ff:fe8c:2a1]%ens33:53           [::]:*    users:(("named",pid=1237,fd=70))
udp   UNCONN 0      0       [fe80::20c:29ff:fe8c:2a1]%ens33:53           [::]:*    users:(("named",pid=1237,fd=69))
udp   UNCONN 0      0      [fe80::555a:7863:514:db46]%ens34:53           [::]:*    users:(("named",pid=1237,fd=86))
udp   UNCONN 0      0      [fe80::555a:7863:514:db46]%ens34:53           [::]:*    users:(("named",pid=1237,fd=85))
udp   UNCONN 0      0      [fe80::555a:7863:514:db46]%ens34:53           [::]:*    users:(("named",pid=1237,fd=87))
udp   UNCONN 0      0      [fe80::555a:7863:514:db46]%ens34:53           [::]:*    users:(("named",pid=1237,fd=84))
tcp   LISTEN 0      10                          127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=54))
tcp   LISTEN 0      10                          127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=57))
tcp   LISTEN 0      10                          127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=55))
tcp   LISTEN 0      10                          127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=52))
```

This step verifies that the DNS server is listening on port 53 for both TCP/UDP protocols on IPv4 and IPv6 [11]. A list of all TCP/UDP listening sockets on the system was listed using ss -tulpn, and entries on port 53 were filtered using grep :53 to ensure that bind9 is actively listening as a DNS service.

Issue 2: Zone Transfer Not Working

Symptoms:

- ns2 slave zone not getting updates
- dig example.lab NS @192.168.100.11 works, but records are old
- New zone file not being created in /var/cache/bind/

Zone transfer (AXFR/IXFR) issues most often:

- allow-transfer config incorrect
- firewall port 53 blocked
- TSIG issue [1], [9], [14]

Troubleshooting Steps:

1. View allow-transfer on primary server

```
hasib@ns1:~$ sudo grep allow-transfer /etc/bind/named.conf.local
    allow-transfer { key ns1-ns2-key; };
    allow-transfer { key ns1-ns2-key; };
```

This step uses grep to quickly verify how allow-transfer is set on the zone. The master server was previously using allow-transfer { 192.168.100.11; }; , but after Task 6, the configuration was updated to allow-transfer { key ns1-ns2-key; }; , which requires TSIG-authenticated zone transfer [9], [14] . This check ensures that ns2 would never have successfully received a zone transfer if the IP or key name were incorrect.

2. Firewall Rule Check

```
hasib@ns1:~$ sudo ufw status
Status: inactive
```

```
hasib@ns1:~$ sudo ufw allow from 192.168.100.11 to any port 53
Rules updated
```

In this step, ufw status is used to verify whether UFW (Uncomplicated Firewall) is enabled and which rules are in effect. DNS zone transfers are usually done over TCP port 53 (UDP is also used if necessary), so it is confirmed that the firewall rule on ns1 allows traffic from ns2's IP 192.168.100.11 to port 53 [11], [9]. This test confirms that the zone transfer would never have been established without the correct rule on the firewall.

3. Force re-transfer

```
hasib@ns2:~$ sudo rndc retransfer example.lab
```

```
hasib@ns2:~$ sudo rndc retransfer example.lab
```

rndc is used as a remote administrative control tool for BIND, and the rndc retransfer command instructs the slave server to re-transfer a fresh zone from the master [1], [31]. As a result, if there is a DNS serial mismatch or outdated zone data, the slave will re-sync with the new data, ensuring zone consistency.

4. Checking Transfer Logs

```
hasib@ns2:~$ sudo grep "transfer" /var/log/syslog | tail -20
grep: /var/log/syslog: binary file matches
2025-11-21T18:29:49.933051+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer completed: 0 messages, 0 records, 0 bytes, 21.034 secs (0 bytes/sec) (serial 0)
2025-11-21T19:49:15.333113+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: failed to connect: connection refused
2025-11-21T19:49:15.333242+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer status: connection refused
2025-11-21T19:49:15.333275+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer completed: 0 messages, 0 records, 0 bytes, 21.047 secs (0 bytes/sec) (serial 0)
2025-11-21T19:56:58.331822+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: failed to connect: connection refused
2025-11-21T19:56:58.332424+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer status: connection refused
2025-11-21T19:56:58.332901+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer completed: 0 messages, 0 records, 0 bytes, 21.039 secs (0 bytes/sec) (serial 0)
2025-11-21T20:02:14.378222+03:00 ns1 kernel: pci 0000:00:00.0: Limiting direct PCI/PCI transfers
2025-11-21T20:48:08.482336+03:00 ns1 kernel: pci 0000:00:00.0: Limiting direct PCI/PCI transfers
2025-11-21T20:51:36.497943+03:00 ns2 kernel: pci 0000:00:00.0: Limiting direct PCI/PCI transfers
2025-11-21T20:56:06.147654+03:00 ns2 named[4881]: transfer of 'example.lab/IN' from 192.168.100.10#53: connected using 1
92.168.100.10#53
2025-11-21T20:56:06.149501+03:00 ns2 named[4881]: zone example.lab/IN: transferred serial 2025111301
2025-11-21T20:56:06.149579+03:00 ns2 named[4881]: transfer of 'example.lab/IN' from 192.168.100.10#53: Transfer status:
success
2025-11-21T20:56:06.149647+03:00 ns2 named[4881]: transfer of 'example.lab/IN' from 192.168.100.10#53: Transfer complete
d: 1 messages, 15 records, 351 bytes, 0.001 secs (351000 bytes/sec) (serial 2025111301)
2025-11-21T20:56:06.647761+03:00 ns2 named[4881]: transfer of '100.168.192.in-addr.arpa/IN' from 192.168.100.10#53: conn
ected using 192.168.100.10#53
2025-11-21T20:56:06.649342+03:00 ns2 named[4881]: zone 100.168.192.in-addr.arpa/IN: transferred serial 2025111301
2025-11-21T20:56:06.649426+03:00 ns2 named[4881]: transfer of '100.168.192.in-addr.arpa/IN' from 192.168.100.10#53: Tran
sfer status: success
```

Zone transfer-related log messages such as Transfer completed are available [11], [14].

5. ACL check

```
hasib@ns1:~$ sudo grep allow-query /etc/bind/named.conf.options
    allow-query { trusted; };
```

```
hasib@ns2:~$ sudo grep allow-query /etc/bind/named.conf.options
    allow-query { trusted; };
```

```
hasib@ns3:~$ sudo grep allow-query /etc/bind/named.conf.options
    allow-query { trusted; };
```

The allow-query checks whether the subnet in which ns1, ns2, ns3 are located is included in the trusted ACL.

6. Firewall (iptables) check

```
hasib@ns1:~$ sudo iptables -L -n | grep 53
hasib@ns1:~$
```

```
hasib@ns2:~$ sudo iptables -L -n | grep 53
hasib@ns2:~$
```

```
hasib@ns3:~$ sudo iptables -L -n | grep 53
hasib@ns3:~$
```

It appears that port 53 is not being dropped in the raw iptables rules.

7. Server connectivity test

```
hasib@ns2:~$ ping 192.168.100.10
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.
64 bytes from 192.168.100.10: icmp_seq=1 ttl=64 time=2.01 ms
64 bytes from 192.168.100.10: icmp_seq=2 ttl=64 time=2.68 ms
64 bytes from 192.168.100.10: icmp_seq=3 ttl=64 time=2.18 ms
64 bytes from 192.168.100.10: icmp_seq=4 ttl=64 time=2.56 ms
64 bytes from 192.168.100.10: icmp_seq=5 ttl=64 time=2.74 ms
64 bytes from 192.168.100.10: icmp_seq=6 ttl=64 time=2.78 ms
64 bytes from 192.168.100.10: icmp_seq=7 ttl=64 time=2.66 ms
64 bytes from 192.168.100.10: icmp_seq=8 ttl=64 time=2.33 ms
64 bytes from 192.168.100.10: icmp_seq=9 ttl=64 time=2.54 ms
64 bytes from 192.168.100.10: icmp_seq=10 ttl=64 time=2.53 ms
64 bytes from 192.168.100.10: icmp_seq=11 ttl=64 time=0.785 ms
64 bytes from 192.168.100.10: icmp_seq=12 ttl=64 time=2.53 ms
64 bytes from 192.168.100.10: icmp_seq=13 ttl=64 time=1.06 ms
64 bytes from 192.168.100.10: icmp_seq=14 ttl=64 time=2.67 ms
64 bytes from 192.168.100.10: icmp_seq=15 ttl=64 time=3.33 ms
64 bytes from 192.168.100.10: icmp_seq=16 ttl=64 time=2.49 ms
64 bytes from 192.168.100.10: icmp_seq=17 ttl=64 time=2.96 ms
64 bytes from 192.168.100.10: icmp_seq=18 ttl=64 time=0.972 ms
```

Ping works, zone transfer is working, TCP 53 connection is established.

Issue 3: Delegation Not Resolving

Symptoms:

- dev1.dept.example.lab is not resolving
- SERVFAIL / NXDOMAIN when querying from ns1
- ns3 may be running fine, but parent delegation is incorrect [2], [9].

Troubleshooting Steps:

1. Verify NS record in parent zone

```
hasib@ns1:~$ dig @192.168.100.10 dept.example.lab NS

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 dept.example.lab NS
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 17323
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b3123dac7c6ba17d010000006921043dab5b2dca09775953 (good)
;; QUESTION SECTION:
;dept.example.lab.              IN      NS

;; AUTHORITY SECTION:
dept.example.lab.       86400   IN      NS      ns3.dept.example.lab.

;; Query time: 3 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 03:30:53 MSK 2025
;; MSG SIZE  rcvd: 91
```

```
hasib@ns2:~$ dig @192.168.100.10 dept.example.lab NS

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 dept.example.lab NS
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 38699
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 4bd2e8e623155305010000000692104c970af6310d2af5020 (good)
;; QUESTION SECTION:
;dept.example.lab.              IN      NS

;; AUTHORITY SECTION:
dept.example.lab.       86400   IN      NS      ns3.dept.example.lab.

;; Query time: 0 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 03:33:13 MSK 2025
;; MSG SIZE  rcvd: 91
```

```
tcp   LISTEN 0      10                                 [::1]:53            [::]:*    users:(("named",pid=1237,fd=65))

hasib@ns3:~$ dig @192.168.100.10 dept.example.lab NS

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 dept.example.lab NS
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4405
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 8fc32d3451210455010000006921045 18e4c6bd4ef357475 (good)
;; QUESTION SECTION:
;dept.example.lab.              IN      NS

;; AUTHORITY SECTION:
dept.example.lab.       86400   IN      NS      ns3.dept.example.lab.

;; Query time: 3 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 03:31:13 MSK 2025
;; MSG SIZE  rcvd: 91
```

At this stage, ns1 is asked who the NS of dept.example.lab is. The output is dept.example.lab. IN NS ns3.dept.example.lab.

2. Glue Record verify

```
hasib@ns1:~$ dig @192.168.100.10 ns3.example.lab A

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 ns3.example.lab A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 13120
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 4e1e1b9a6d12a1ab010000006921049cab6c15a70354decb (good)
;; QUESTION SECTION:
;ns3.example.lab.               IN      A

;; ANSWER SECTION:
ns3.example.lab.        86400   IN      A       192.168.100.12

;; Query time: 0 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 03:32:28 MSK 2025
;; MSG SIZE  rcvd: 88
```

```
hasib@ns2:~$ dig @192.168.100.10 ns3.example.lab A

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 ns3.example.lab A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 1695
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 8fe6442932e917a4010000006921046cb5e62523f2aadd2f (good)
;; QUESTION SECTION:
;ns3.example.lab.               IN      A

;; ANSWER SECTION:
ns3.example.lab.        86400   IN      A       192.168.100.12

;; Query time: 0 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 03:31:40 MSK 2025
;; MSG SIZE  rcvd: 88
```

```
hasib@ns3:~$ dig @192.168.100.10 ns3.example.lab A

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 ns3.example.lab A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6127
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 46bd4ba17f4232c3010000006921048c36c26f80e8e90798 (good)
;; QUESTION SECTION:
;ns3.example.lab.               IN      A

;; ANSWER SECTION:
ns3.example.lab.        86400   IN      A       192.168.100.12

;; Query time: 2 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 03:32:12 MSK 2025
;; MSG SIZE  rcvd: 88
```

This step is to see what is in Parent. Glue record BOOM important It gives the resolver the IP of NS for delegation to work [32], [33]. It is correct and is found: ns3.dept.example.lab. IN A 192.168.100.12

3. Delegated server authoritative or not

```
hasib@ns3:~$ dig @192.168.100.12 dept.example.lab SOA

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.12 dept.example.lab SOA
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29269
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: e50826fe0a400db20100000692104dee8e8a7690e695c31 (good)
;; QUESTION SECTION:
;dept.example.lab.              IN      SOA

;; ANSWER SECTION:
dept.example.lab.       86400   IN      SOA     ns3.dept.example.lab. admin.example.lab. 2025111301 3600 1800 604800 864
00

;; Query time: 42 msec
;; SERVER: 192.168.100.12#53(192.168.100.12) (UDP)
;; WHEN: Sat Nov 22 03:33:34 MSK 2025
;; MSG SIZE  rcvd: 119
```

At this point, the query for the SOA record has been sent directly to ns, and as expected, ns3 returned an authoritative SOA response, confirming that it is functioning properly as the authoritative name server for dept.example.lab.

4. Full trace test

```
hasib@ns2:~$

dig @192.168.100.10 hr.dept.example.lab +trace

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 hr.dept.example.lab +trace
; (1 server found)
;; global options: +cmd
;; Received 56 bytes from 192.168.100.10#53(192.168.100.10) in 1 ms
```

```
hasib@ns3:~$ dig @192.168.100.10 hr.dept.example.lab +trace

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 hr.dept.example.lab +trace
; (1 server found)
;; global options: +cmd
;; Received 56 bytes from 192.168.100.10#53(192.168.100.10) in 2 ms

hasib@ns3:~$
```

Using +trace dig displays the entire DNS resolution path, showing which server was queried at each step [28]. Here we see that ns1 first finds the NS record for dept.example.lab, then queries ns3 based on that information to obtain the final authoritative answer.

Issue 4: DNS Queries Timing Out

Timeout means that the client is sending a query, but no response is coming (UDP drop, firewall, listen-on error, etc.) [2][7].

At this stage-

- Is the DNS server listening on port 53 or not?
- Is the server reachable or not?
- Is the allow-query, firewall, iptables config correct or not?
- Is BIND running or not? [1], [9], [11]

1. Check BIND Listening on Port 53

```
hasib@ns3:~$ sudo ss -tulpn | grep :53
udp   UNCONN 0      0                                0.0.0.0:5353       0.0.0.0:*    users:(("avahi-daemon",pid=938,fd=12
))
udp   UNCONN 0      0                        192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=92))
udp   UNCONN 0      0                        192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=94))
udp   UNCONN 0      0                        192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=93))
udp   UNCONN 0      0                        192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=95))
udp   UNCONN 0      0                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=51))
udp   UNCONN 0      0                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=50))
udp   UNCONN 0      0                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=49))
udp   UNCONN 0      0                             127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=48))
udp   UNCONN 0      0                            127.0.0.54:53         0.0.0.0:*    users:(("systemd-resolve",pid=606,fd
=16))
udp   UNCONN 0      0                         127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=606,fd
=14))
udp   UNCONN 0      0                                  [::]:5353          [::]:*    users:(("avahi-daemon",pid=938,fd=13
))
udp   UNCONN 0      0                                 [::1]:53            [::]:*    users:(("named",pid=1237,fd=60))
udp   UNCONN 0      0                                 [::1]:53            [::]:*    users:(("named",pid=1237,fd=61))
```

```
tcp   LISTEN 0      10                            127.0.0.1:53         0.0.0.0:*    users:(("named",pid=1237,fd=52))
tcp   LISTEN 0      4096                       127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=606,fd
=15))
tcp   LISTEN 0      4096                          127.0.0.54:53         0.0.0.0:*    users:(("systemd-resolve",pid=606,fd
=17))
tcp   LISTEN 0      10                           192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=97))
tcp   LISTEN 0      10                           192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=98))
tcp   LISTEN 0      10                           192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=99))
tcp   LISTEN 0      10                           192.168.100.12:53         0.0.0.0:*    users:(("named",pid=1237,fd=96))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=73))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=72))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=74))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53            [::]:*    users:(("named",pid=1237,fd=75))
tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=89))
tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=91))
tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=90))
tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53            [::]:*    users:(("named",pid=1237,fd=88))
```

```
tcp   LISTEN 0      10                     192.168.100.12:53          0.0.0.0:*    users:(("named",pid=1237,fd=99))
tcp   LISTEN 0      10                     192.168.100.12:53          0.0.0.0:*    users:(("named",pid=1237,fd=96))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53             [::]:*    users:(("named",pid=1237,fd=73))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53             [::]:*    users:(("named",pid=1237,fd=72))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53             [::]:*    users:(("named",pid=1237,fd=74))
tcp   LISTEN 0      10      [fe80::20c:29ff:fe8c:2a1]%ens33:53             [::]:*    users:(("named",pid=1237,fd=75))
tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53             [::]:*    users:(("named",pid=1237,fd=89))
tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53             [::]:*    users:(("named",pid=1237,fd=91))
tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53             [::]:*    users:(("named",pid=1237,fd=90))
tcp   LISTEN 0      10     [fe80::555a:7863:514:db46]%ens34:53             [::]:*    users:(("named",pid=1237,fd=88))
tcp   LISTEN 0      10                                [::1]:53             [::]:*    users:(("named",pid=1237,fd=67))
tcp   LISTEN 0      10                                [::1]:53             [::]:*    users:(("named",pid=1237,fd=66))
tcp   LISTEN 0      10                                [::1]:53             [::]:*    users:(("named",pid=1237,fd=64))
tcp   LISTEN 0      10                                [::1]:53             [::]:*    users:(("named",pid=1237,fd=65))
```

```
udp   UNCONN 0      0                                 [::1]:53             [::]:*    users:(("named",pid=1237,fd=61))
udp   UNCONN 0      0                                 [::1]:53             [::]:*    users:(("named",pid=1237,fd=62))
udp   UNCONN 0      0                                 [::1]:53             [::]:*    users:(("named",pid=1237,fd=63))
udp   UNCONN 0      0       [fe80::20c:29ff:fe8c:2a1]%ens33:53             [::]:*    users:(("named",pid=1237,fd=68))
udp   UNCONN 0      0       [fe80::20c:29ff:fe8c:2a1]%ens33:53             [::]:*    users:(("named",pid=1237,fd=71))
udp   UNCONN 0      0       [fe80::20c:29ff:fe8c:2a1]%ens33:53             [::]:*    users:(("named",pid=1237,fd=70))
udp   UNCONN 0      0       [fe80::20c:29ff:fe8c:2a1]%ens33:53             [::]:*    users:(("named",pid=1237,fd=69))
udp   UNCONN 0      0      [fe80::555a:7863:514:db46]%ens34:53             [::]:*    users:(("named",pid=1237,fd=86))
udp   UNCONN 0      0      [fe80::555a:7863:514:db46]%ens34:53             [::]:*    users:(("named",pid=1237,fd=85))
udp   UNCONN 0      0      [fe80::555a:7863:514:db46]%ens34:53             [::]:*    users:(("named",pid=1237,fd=87))
udp   UNCONN 0      0      [fe80::555a:7863:514:db46]%ens34:53             [::]:*    users:(("named",pid=1237,fd=84))
tcp   LISTEN 0      10                           127.0.0.1:53           0.0.0.0:*    users:(("named",pid=1237,fd=54))
tcp   LISTEN 0      10                           127.0.0.1:53           0.0.0.0:*    users:(("named",pid=1237,fd=57))
tcp   LISTEN 0      10                           127.0.0.1:53           0.0.0.0:*    users:(("named",pid=1237,fd=55))
tcp   LISTEN 0      10                           127.0.0.1:53           0.0.0.0:*    users:(("named",pid=1237,fd=52))
```

This step verifies that the DNS server is listening on port 53 for both TCP/UDP protocols on IPv4 and IPv6 [11]. A list of all TCP/UDP listening sockets on the system was listed using ss -tulpn, and entries on port 53 were filtered using grep :53 to ensure that bind9 is actively listening as a DNS service.

2. Server connectivity test

```
hasib@ns2:~$ ping 192.168.100.10
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.
64 bytes from 192.168.100.10: icmp_seq=1 ttl=64 time=2.01 ms
64 bytes from 192.168.100.10: icmp_seq=2 ttl=64 time=2.68 ms
64 bytes from 192.168.100.10: icmp_seq=3 ttl=64 time=2.18 ms
64 bytes from 192.168.100.10: icmp_seq=4 ttl=64 time=2.56 ms
64 bytes from 192.168.100.10: icmp_seq=5 ttl=64 time=2.74 ms
64 bytes from 192.168.100.10: icmp_seq=6 ttl=64 time=2.78 ms
64 bytes from 192.168.100.10: icmp_seq=7 ttl=64 time=2.66 ms
64 bytes from 192.168.100.10: icmp_seq=8 ttl=64 time=2.33 ms
64 bytes from 192.168.100.10: icmp_seq=9 ttl=64 time=2.54 ms
64 bytes from 192.168.100.10: icmp_seq=10 ttl=64 time=2.53 ms
64 bytes from 192.168.100.10: icmp_seq=11 ttl=64 time=0.785 ms
64 bytes from 192.168.100.10: icmp_seq=12 ttl=64 time=2.53 ms
64 bytes from 192.168.100.10: icmp_seq=13 ttl=64 time=1.06 ms
64 bytes from 192.168.100.10: icmp_seq=14 ttl=64 time=2.67 ms
64 bytes from 192.168.100.10: icmp_seq=15 ttl=64 time=3.33 ms
64 bytes from 192.168.100.10: icmp_seq=16 ttl=64 time=2.49 ms
64 bytes from 192.168.100.10: icmp_seq=17 ttl=64 time=2.96 ms
64 bytes from 192.168.100.10: icmp_seq=18 ttl=64 time=0.972 ms
```

Ping works, zone transfer is working, TCP 53 connection is established.

3. ACL check

```
hasib@ns1:~$ sudo grep allow-query /etc/bind/named.conf.options
    allow-query { trusted; };
```

```
hasib@ns2:~$ sudo grep allow-query /etc/bind/named.conf.options
    allow-query { trusted; };
```

```
hasib@ns3:~$ sudo grep allow-query /etc/bind/named.conf.options
    allow-query { trusted; };
```

The subnet in the allow-query is checked to see if it is in the trusted ACL.

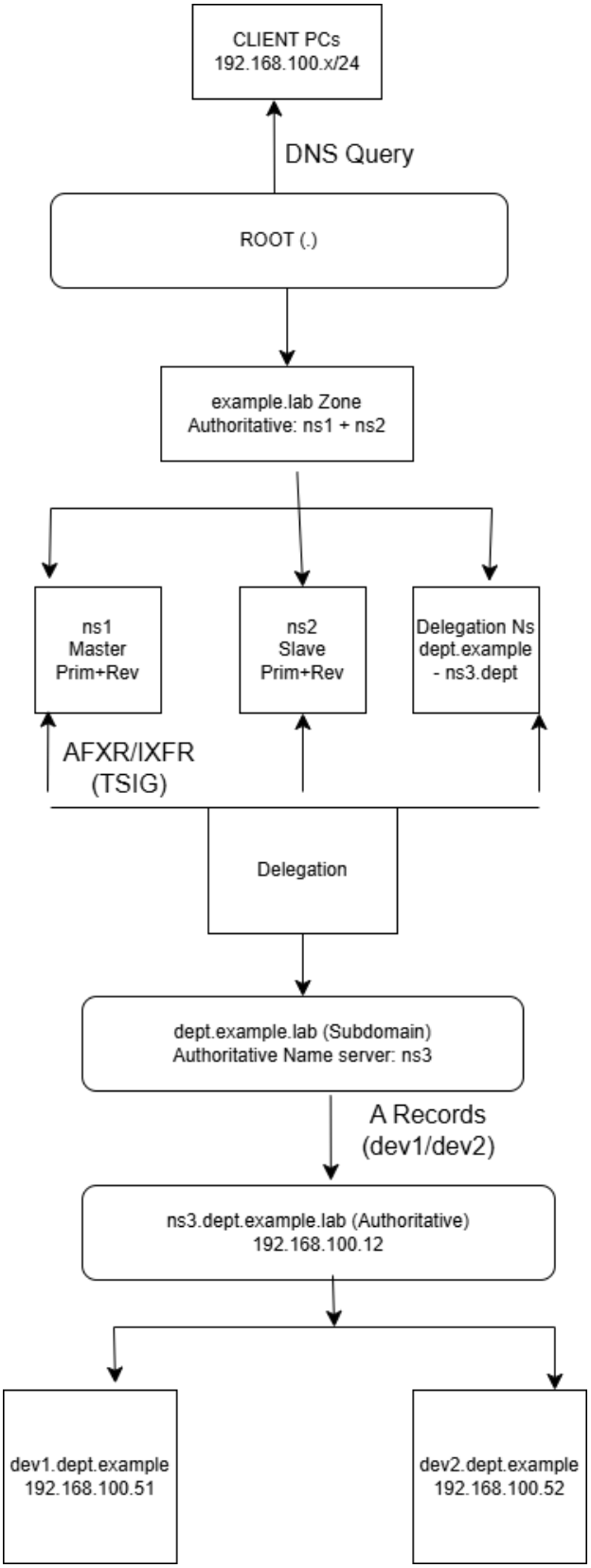
4. Firewall (iptables) check

It appears that port 53 is not being dropped in the raw iptables rules.

```
hasib@ns1:~$ sudo iptables -L -n | grep 53
hasib@ns1:~$
```

```
hasib@ns2:~$ sudo iptables -L -n | grep 53
hasib@ns2:~$
```

❖ **NETWORK DIAGRAM**



ns1 (192.168.100.10)

Primary Master DNS

- example.lab zone
- Reverse zone
- Delegation NS + Glue record added
- Sends AXFR/IXFR to ns2 (TSIG-secured)

ns2 (192.168.100.11)

Secondary Slave DNS

- Takes Zone Transfer from ns1
- Serves example.lab + Reverse zone
- Gives authoritative answer if ns1 is down

ns3 (192.168.100.12)

Delegated Authoritative DNS

- dept.example.lab zone
- Keeps A records for dev1, dev2
- Receives Delegation + Glue from parent

Task 8: Questions and Analysis

Question 1: Explain the difference between authoritative and recursive DNS servers. Which type did you configure in this lab?

Answer: There are two types of DNS servers - Authoritative and Recursive.

An Authoritative DNS Server is a server that holds the original (official) information for a specific zone and provides that information to the client. An Authoritative Server never sends requests to other servers, but rather responds directly from the A, PTR, MX, NS, SOA records stored in the zone file [9]. On the other hand, a Recursive DNS Server completes the entire DNS resolution process on behalf of the client - first to the root server, then to the TLD server, then to the authoritative server, and finally collects the IP and gives it to the client [10], [11].

The DNS servers that we have configured in this lab, such as ns1, ns2, and ns3, are all Authoritative-Only Servers. This is because recursion no; is explicitly set in the named.conf.options file. That is, the servers do not resolve recursive queries, but only provide authoritative answers for their own zones.

Question 2: What is the purpose of the Serial number in the SOA record? What happens if you forget to increment it when making zone changes

Answer: The SOA (Start of Authority) record carries the most important metadata of a DNS zone, one of which is the Serial Number. The Serial Number tells the slave server or secondary server whether the zone file of the master server has been updated. Whenever the zone file is changed on the master server (ns1), the Serial Number must be incremented. Without proper Serial increment, the secondary server (ns2) will never know that the zone file has been updated, and will not receive the new information [1], [5]. This will result in zone transfers being disrupted and old DNS records will remain on the slave server - which causes a major problem in a production environment, as clients may receive stale/old records.

Question 3: Describe the zone transfer process. What are the differences between AXFR and IXFR?

Answer: Zone Transfer is the process of sending zone data from the master DNS server (ns1) to the slave server (ns2), so that the secondary server is always updated. There are two types of Zone Transfer:

1. AXFR (Full Zone Transfer) - In this, the entire zone file is sent to the slave server at once. AXFR is used when the zone is small or is a first-time transfer [6].

2. IXFR (Incremental Zone Transfer) - In this, only the changed records are sent to the slave server. It is bandwidth-efficient and fast, especially for large zones [7].

The test performed in this lab (dig @ns1 example.lab AXFR) is an AXFR transfer.

Question 4: Why are glue records necessary in DNS delegation? What problems occur without them??

Answer: DNS delegation is the process of delegating responsibility from the parent zone to the child zone. However, if the parent zone does not know the IP address of the delegated name server, a circular dependency or infinite loop is created.

A Glue Record is an A record placed in the parent zone (example.lab) that tells the IP address of the delegated name server (such as ns3.dept.example.lab) [6], [7], [12].

If there is no Glue Record, the following problems occur:

• The parent zone shows the NS record and says: “Ask ns3.”

• Client/ns1 says: “Where is ns3? What is the IP?”

• Parent says: “I don’t know.” → Loop begins

• As a result, query resolution fails (SERVFAIL/NXDOMAIN).

A Glue Record is required for subdomain delegation to succeed.

Question 5: What security risks exist with improperly configured zone transfers? How did you mitigate these risks in your configuration?

Answer: If Zone Transfer is misconfigured (e.g. allow-transfer { any; };), an attacker can download the entire zone file, which is known as a DNS reconnaissance attack [7], [19]. This exposes the internal server name, IP mapping, mail server details, subdomains, etc. - the initial step in a cyber-attack.

In this lab, to mitigate the risk:

TSIG (HMAC-SHA256) authentication is used.

- **On ns1:**

```
hasib@ns1:~$ sudo tsig-keygen -a hmac-sha256 ns1-ns2-key
[sudo] password for hasib:
key "ns1-ns2-key" {
        algorithm hmac-sha256;
        secret "K/ok7RRnRCrHNubNJEe3aSOqmq22hnwRQW3bjaz/xVw=";
};
```

- **On ns2:**

```
hasib@ns2:~$ sudo nano /etc/bind/named.conf.local
```

```
zone "example.lab" {
    type slave;
    file "/var/cache/bind/db.example.lab";
    masters { 192.168.100.10 key ns1-ns2-key; };
};
```

Zone transfers are completely prohibited except for this key-exactly 2 keys are the best method of zone transfer authentication.

Question 6: If the primary DNS server fails, can clients still resolve names in the zone? Explain why or why not.

Answer: Yes, clients can still resolve DNS if the Primary DNS (ns1) fails, because the secondary server (ns2) holds a complete copy of the authoritative zone.

There were two servers listed in the DNS NS record

example.lab NS ns1.example.lab.

```
hasib@ns1:~$ sudo cat /etc/bind/zones/db.example.lab
[sudo] password for hasib:
$TTL 86400
@   IN  SOA ns1.example.lab. admin.example.lab. (
        2025111301 ; Serial (YYYYMMDDNN format)
        3600       ; Refresh (1 hour)
        1800       ; Retry (30 minutes)
        604800     ; Expire (1 week)
        86400 )    ; Negative Cache TTL (1 day)

; Name Servers
@   IN  NS  ns1.example.lab.
@   IN  NS  ns2.example.lab.

; A Records for Name Servers
ns1 IN  A   192.168.100.10
ns2 IN  A   192.168.100.11

; Mail Exchange Record
@   IN  MX  10 mail.example.lab.

; Host A Records
www IN  A   192.168.100.20
mail IN A   192.168.100.25
ftp  IN A   192.168.100.30
db   IN A   192.168.100.35
```

```
; Alias (CNAME) Records
webmail IN CNAME www.example.lab.
smtp    IN CNAME mail.example.lab.

; Delegation for subdomain
dept IN NS ns3.dept.example.lab.
ns3  IN A  192.168.100.12
```

example.lab NS ns2.example.lab.

```
hasib@ns2:~$ sudo cat /etc/bind/zones/db.example.lab
$TTL 86400
@   IN  SOA ns1.example.lab. admin.example.lab. (
        2025111301 ; Serial (YYYYMMDDNN format)
        3600       ; Refresh (1 hour)
        1800       ; Retry (30 minutes)
        604800     ; Expire (1 week)
        86400 )    ; Negative Cache TTL (1 day)

; Name Servers
@   IN  NS  ns1.example.lab.
@   IN  NS  ns2.example.lab.

; A Records for Name Servers
ns1 IN  A   192.168.100.10
ns2 IN  A   192.168.100.11

; Mail Exchange Record
@   IN  MX  10 mail.example.lab.

; Host A Records
www IN  A   192.168.100.20
mail IN A   192.168.100.25
ftp  IN A   192.168.100.30
db   IN A   192.168.100.35
```

```
; Alias (CNAME) Records
webmail IN CNAME www.example.lab.
smtp    IN CNAME mail.example.lab.

; Delegation for subdomain
dept IN NS ns3.example.lab.
ns3  IN A  192.168.100.12
```

According to the RFC, recursive resolvers fall back to the secondary server if the primary is unavailable when multiple NS records are found [8]. Therefore, ns2 is able to serve the entire zone and DNS resolution is uninterrupted.

Question 7: Describe three methods you used to troubleshoot DNS issues during this lab.

Answer: Three main problem analysis methods were used in this lab:

(1) named-checkconf / named-checkzone

Config ও zone file syntax error চেক করার জন্য:

named-checkconf

```
hasib@ns1:~$ sudo named-checkconf
```

```
hasib@ns2:~$ sudo named-checkconf
```

```
hasib@ns3:~$ sudo named-checkconf
```

named-checkzone example.lab db.example.lab

```
hasib@ns1:~$ sudo named-checkzone example.lab /etc/bind/zones/db.example.lab
zone example.lab/IN: dept.example.lab/NS 'ns3.dept.example.lab' has no REQUIRED GLUE address records (A or AAAA)
zone example.lab/IN: dept.example.lab/NS 'ns3.dept.example.lab' (out of zone) has no addresses records (A or AAAA)
zone example.lab/IN: loaded serial 2025111301
OK
```

```
hasib@ns3:~$ sudo named-checkzone dept.example.lab /etc/bind/zones/db.dept.example.lab
zone dept.example.lab/IN: loaded serial 2025111301
OK
```

(2) journalctl / syslog debugging

To detect service failure, TSIG error, permission issue:

```
hasib@ns1:~$ sudo journalctl -xeu bind9
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
-- No entries --
```

```
hasib@ns2:~$ sudo journalctl -xeu bind9
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
-- No entries --
```

```
hasib@ns3:~$ sudo journalctl -xeu bind9
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
-- No entries --
```

grep "transfer" /var/log/syslog

```
hasib@ns2:~$ sudo grep "transfer" /var/log/syslog | tail -20
grep: /var/log/syslog: binary file matches
2025-11-21T18:29:49.933051+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer completed: 0 messages, 0 records, 0 bytes, 21.034 secs (0 bytes/sec) (serial 0)
2025-11-21T19:49:15.333113+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: failed to connect: connection refused
2025-11-21T19:49:15.333242+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer status: connection refused
2025-11-21T19:49:15.333275+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer completed: 0 messages, 0 records, 0 bytes, 21.047 secs (0 bytes/sec) (serial 0)
2025-11-21T19:56:58.331822+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: failed to connect: connection refused
2025-11-21T19:56:58.332424+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer status: connection refused
2025-11-21T19:56:58.332901+03:00 hasib-VMware-Virtual-Platform named[1242]: transfer of 'example.lab/IN' from 192.168.10
0.10#53: Transfer completed: 0 messages, 0 records, 0 bytes, 21.039 secs (0 bytes/sec) (serial 0)
2025-11-21T20:02:14.378222+03:00 ns1 kernel: pci 0000:00:00.0: Limiting direct PCI/PCI transfers
2025-11-21T20:48:08.482336+03:00 ns1 kernel: pci 0000:00:00.0: Limiting direct PCI/PCI transfers
2025-11-21T20:51:36.497943+03:00 ns2 kernel: pci 0000:00:00.0: Limiting direct PCI/PCI transfers
2025-11-21T20:56:06.147654+03:00 ns2 named[4881]: transfer of 'example.lab/IN' from 192.168.100.10#53: connected using 1
92.168.100.10#53
2025-11-21T20:56:06.149501+03:00 ns2 named[4881]: zone example.lab/IN: transferred serial 2025111301
2025-11-21T20:56:06.149579+03:00 ns2 named[4881]: transfer of 'example.lab/IN' from 192.168.100.10#53: Transfer status:
success
2025-11-21T20:56:06.149647+03:00 ns2 named[4881]: transfer of 'example.lab/IN' from 192.168.100.10#53: Transfer complete
d: 1 messages, 15 records, 351 bytes, 0.001 secs (351000 bytes/sec) (serial 2025111301)
2025-11-21T20:56:06.647761+03:00 ns2 named[4881]: transfer of '100.168.192.in-addr.arpa/IN' from 192.168.100.10#53: conn
ected using 192.168.100.10#53
2025-11-21T20:56:06.649342+03:00 ns2 named[4881]: zone 100.168.192.in-addr.arpa/IN: transferred serial 2025111301
2025-11-21T20:56:06.649426+03:00 ns2 named[4881]: transfer of '100.168.192.in-addr.arpa/IN' from 192.168.100.10#53: Tran
sfer status: success
```

(3) dig diagnostics (A, NS, PTR, AXFR, +trace)

To resolve delegation error, zone transfer problem, glue record problem:

dig dept.example.lab +trace

```
hasib@ns2:~$

dig @192.168.100.10 hr.dept.example.lab +trace

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 hr.dept.example.lab +trace
; (1 server found)
;; global options: +cmd
;; Received 56 bytes from 192.168.100.10#53(192.168.100.10) in 1 ms
```

```
hasib@ns3:~$ dig @192.168.100.10 hr.dept.example.lab +trace

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 hr.dept.example.lab +trace
; (1 server found)
;; global options: +cmd
;; Received 56 bytes from 192.168.100.10#53(192.168.100.10) in 2 ms

hasib@ns3:~$
```

dig example.lab NS

```
hasib@ns1:~$ dig @192.168.100.10 dept.example.lab NS

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 dept.example.lab NS
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 17323
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b3123dac7c6ba17d010000006921043dab5b2dca09775953 (good)
;; QUESTION SECTION:
;dept.example.lab.              IN      NS

;; AUTHORITY SECTION:
dept.example.lab.       86400   IN      NS      ns3.dept.example.lab.

;; Query time: 3 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 03:30:53 MSK 2025
;; MSG SIZE  rcvd: 91
```

```
hasib@ns2:~$ dig @192.168.100.10 dept.example.lab NS

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 dept.example.lab NS
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 38699
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 4bd2e8e623155305010000006921 04c970af6310d2af5020 (good)
;; QUESTION SECTION:
;dept.example.lab.              IN      NS

;; AUTHORITY SECTION:
dept.example.lab.       86400   IN      NS      ns3.dept.example.lab.

;; Query time: 0 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 03:33:13 MSK 2025
;; MSG SIZE  rcvd: 91
```

```
tcp   LISTEN 0      10                             [::1]:53           [::]:*    users:(("named",pid=1237,fd=65))

hasib@ns3:~$ dig @192.168.100.10 dept.example.lab NS

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 dept.example.lab NS
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4405
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 8fc32d3451210455010000006921045 18e4c6bd4ef357475 (good)
;; QUESTION SECTION:
;dept.example.lab.              IN      NS

;; AUTHORITY SECTION:
dept.example.lab.       86400   IN      NS      ns3.dept.example.lab.

;; Query time: 3 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Sat Nov 22 03:31:13 MSK 2025
;; MSG SIZE  rcvd: 91
```

dig @ns2 example.lab AXFR

```
hasib@ns2:~$ dig @192.168.100.10 example.lab AXFR -k /etc/bind/tsig-keys.conf

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.100.10 example.lab AXFR -k /etc/bind/tsig-keys.conf
; (1 server found)
;; global options: +cmd
example.lab.            86400   IN      SOA     ns1.example.lab. admin.example.lab. 2025111301 3600 1800 604800 86400
example.lab.            86400   IN      NS      ns1.example.lab.
example.lab.            86400   IN      NS      ns2.example.lab.
example.lab.            86400   IN      MX      10 mail.example.lab.
db.example.lab.         86400   IN      A       192.168.100.35
dept.example.lab.       86400   IN      NS      ns3.dept.example.lab.
ftp.example.lab.        86400   IN      A       192.168.100.30
mail.example.lab.       86400   IN      A       192.168.100.25
ns1.example.lab.        86400   IN      A       192.168.100.10
ns2.example.lab.        86400   IN      A       192.168.100.11
ns3.example.lab.        86400   IN      A       192.168.100.12
smtp.example.lab.       86400   IN      CNAME   mail.example.lab.
webmail.example.lab.    86400   IN      CNAME   www.example.lab.
www.example.lab.        86400   IN      A       192.168.100.20
example.lab.            86400   IN      SOA     ns1.example.lab. admin.example.lab. 2025111301 3600 1800 604800 86400
ns1-ns2-key.            0       ANY     TSIG    hmac-sha256. 1763769422 300 32 bEDtDv4HJ8Tv26znpQuhbjwm3/t4Dqk2IiJRgI2Eb
cA= 39866 NOERROR 0
;; Query time: 0 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (TCP)
;; WHEN: Sat Nov 22 02:57:02 MSK 2025
;; XFR size: 15 records (messages 1, bytes 478)
```

dig +trace is especially useful for error detection.

Question 8: What is the purpose of TSIG? In what scenarios would you implement it in a production environment?

Answer: TSIG (Transaction Signature) is a secure method of DNS message authentication. It uses HMAC-based shared secret keys to secure communication between DNS servers [19]. TSIG prevents replay attacks, ensures message integrity, and provides authentication for zone transfers or dynamic updates.

TSIG should be used in production for:

• Master–Slave secure zone transfer

• Dynamic DNS update (DDNS)

• Anycast DNS cluster synchronization

• Internal/private DNS zones security

Using TSIG greatly strengthens the authentication layer of the DNS system.

References

[1] DigitalOcean, “How to Configure BIND as a Private Network DNS Server on Ubuntu 14.04.” Available: https://www.digitalocean.com/community/tutorials/how-to-configure-bind-as-a-private-network-dns-server-on-ubuntu-14-04

[2] Bissquit, “Шпаргалка по DNS.” Available: https://blog.bissquit.com/windows/shpargalka-po-dns/

[3] InnoTech, “DNS Maintenance Guide.” Available: https://docs.inno.tech/ru/directory-service/latest/maintenance-guide/dns/

[4] Habr QnA, “AXFR zone transfer - Q&A.” Available: https://qna.habr.com/q/52408

[5] ISC, “BIND9 Administrator Reference Manual - Chapter 4.” Available: https://bind9.readthedocs.io/en/latest/chapter4.html

[6] Acunetix, “DNS Zone Transfers (AXFR) Explained.” Available: https://www.acunetix.com/blog/articles/dns-zone-transfers-axfr/

[7] Veeble, “DNS Zone Transfer Overview.” Available: https://www.veeble.com/kb/dns-zone-transfer/

[8] Dotcom-Monitor, “How to Troubleshoot DNS Errors.” Available: https://www.dotcom-monitor.com/wiki/knowledge-base/how-to-troubleshoot-dns-errors/

[9] Red Hat, “Setting up and Configuring a BIND DNS Server.” Available: https://docs.redhat.com/.../assembly_setting-up-and-configuring-a-bind-dns-server_networking-infrastructure-services

[10] Timeweb Cloud, “Настройка DNS сервера Bind.” Available: https://timeweb.cloud/tutorials/ubuntu/nastrojka-dns-servera-bind

[11] ClouDNS, “Zone Transfer & Zone File Namespace.” Available: https://www.cloudns.net/blog/zone-transfer-zone-file-domain-namespace/

[12] CITForum, “DNS Basics by Khramtsov.” Available: https://citforum.ru/internet/dns/khramtsov/3.shtml

[13] AstraLinux, “DNS Configuration Documentation.” Available: https://wiki.astralinux.ru/pages/viewpage.action?pageId=27362248

[14] ClouDNS, “Master–Slave DNS Explained.” Available: https://www.cloudns.net/blog/master-slave-dns/

[15] StackOverflow RU, “В чем разница между master, slave и secondary DNS?” Available: https://ru.stackoverflow.com/questions/347306/

[16] Litux Documentation, “DNS Reference Chapter.” Available: https://litux.nl/Reference/books/7213/ddu0262.html

[17] IBM, “named-checkconf Command Documentation.” Available: https://www.ibm.com/docs/ru/aix/7.2.0?topic=n-named-checkconf-command

[18] Cilium, “How to Debug DNS Issues in Kubernetes.” Available: https://cilium.io/blog/2019/12/18/how-to-debug-dns-issues-in-k8s/

[19] CIRA, “Using TSIG for DNS Security.” Available: https://www.cira.ca/uploads/2023/05/UsingTSIG-EN.pdf

[20] CherryServers, “How to Install and Configure Private BIND DNS Server.” Available: https://www.cherryservers.com/blog/how-to-install-and-configure-a-private-bind-dns-server-on-ubuntu-22-04

[21] Kubernetes Docs, “DNS Debugging and Resolution.” Available: https://kubernetes.io/docs/tasks/administer-cluster/dns-debugging-resolution/

[22] NSRC, “TSIG Training Material.” Available: https://nsrc.org/.../TSIG-17022011.pdf

[23] Groundcover, “DNS Issues Troubleshooting in Kubernetes.” [Online]. Available: https://www.groundcover.com/kubernetes-troubleshooting/dns-issues

[24] F5 Networks, “BIG-IP DNS Services Implementation.” Available: https://techdocs.f5.com/.../bigip-dns-services-implementations

[25] Cloudflare, “DNS Zone Transfers.” Available: https://developers.cloudflare.com/dns/zone-setups/zone-transfers/

[26] SWeb, “Настройка DNS сервера BIND.” Available: https://help.sweb.ru/nastrojka-dns-servera-bind_1315.html

[27] DNSimple, “Debugging DNS.” Available: https://blog.dnsimple.com/2017/08/debugging-dns/

[28] PowerDNS, “TSIG Documentation.” Available: https://doc.powerdns.com/authoritative/tsig.html

[29] Julia Evans, “How Do You Tell If a Problem Is Caused by DNS?”. Available: https://jvns.ca/blog/2021/11/04/how-do-you-tell-if-a-problem-is-caused-by-dns/

[30] ISC, “DNSSEC Guide.” Available: https://bind9.readthedocs.io/en/latest/dnssec-guide.html

[31] Akamai, “Enable DNSSEC.” Available: https://techdocs.akamai.com/edge-dns/docs/enable-dnssec

[32] ISC, “BIND9 Manual Chapter 3.” Available: https://bind9.readthedocs.io/en/latest/chapter3.html

[33] Microsoft Learn, “Windows DNS Server Q&A.” Available: https://learn.microsoft.com/en-us/answers/questions/281806/windows-dns-server